Assume Breach

A First Course in Defensive Security

Michael Borck

2026-11-04

Assume Breach
A First Course in Defensive Security

Copyright © 2026 Michael Borck.

Book content: Creative Commons Attribution 4.0 International (CC BY 4.0 International).
https://creativecommons.org/licenses/by/4.0/

Original code examples and accompanying software: MIT License.
https://opensource.org/licenses/MIT

These grants cover material the author has authority to license. Separately credited third-party material retains its stated terms.

Published by Michael Borck
Perth, Western Australia

ISBN: (to be assigned)

First edition, in development.

AI disclosure: AI tools were used as thinking partners throughout the drafting, iterating, and refining of this book. The author reviewed, challenged, and took responsibility for every sentence.

Companion website:
https://michael-borck.github.io/assume-breach
Source: https://github.com/michael-borck/assume-breach

Table of contents

Introduction: Assume Breach 1
Acknowledgments 5

I Part I · Foundations 7
1 Introduction to Information Security 9
2 Software Security & Malware 17
3 Authentication & Access 25
4 Cryptography 33

II Part II · Managing Risk 41
5 Risk Management 43
6 Incident & Disaster Planning 51

III Part III · Defending Systems & Networks 59
7 Network Security 61
8 VPN & Firewalls 69
9 Web Security & Data Protection 77

IV Part IV · People & the Evolving Threat 85
10 Cybercrime, Botnets & Forensics 87
11 Human Factors & Emerging Trends 93

V Part V · Putting It Together 99
12 The Long Game 101

Appendices **105**

Appendix: Setting Up a Practice Environment 105

A The Defender's Dossier 109

B Further Reading 119

About the Author 123

Introduction: Assume Breach

A security control can fail. A supplier account can be misused, a patch can arrive late, or a colleague can approve an unexpected login request. A defender plans for those possibilities while still trying to prevent them. **Assume breach** means designing detection, containment and recovery into the system.

Who this book is for

This is a first course for students, people moving into security, and curious professionals. It assumes basic familiarity with a browser and files, but no security background or programming. The core reading and exercises require no software installation, paid service, AI account or group.

By the end, you should be able to take a small system description, identify important assets and plausible threats, choose layered controls within a budget, specify tests of those controls, and defend a response/recovery decision with its uncertainties. This is introductory judgement, not authorisation to test other people's systems or a qualification to run a production incident alone.

How the chapters build

Read in order on your first pass. Each part contributes to a short **defender's dossier**, using the fictional Harbour Community Centre in The Defender's Dossier.

Part	What you should be able to do afterwards
I: Foundations	Identify assets, threats, permissions and appropriate protection mechanisms
II: Managing Risk	Justify a control choice and check a recovery plan against stated targets
III: Systems and Networks	Trace a trust boundary and test permitted and forbidden actions
IV: People and Threats	Separate evidence from inference and design a usable verification process
V: Putting It Together	Defend a complete plan for a changed case and identify what is not ready

Every chapter has questions to consider and a short applied task. Try the task before reading the worked dossier. The final chapter gives an independent variant and success criteria. Definitions alone are not evidence that you can use them.

The network primer is in chapter 7. NTLM, cryptographic attack models and named tools provide optional depth; you do not need to configure them to complete the dossier. The practice appendix explains boundaries for optional hands-on work. It does not claim that a generic VM or container setting creates a safe lab.

Several chapters link *Incident Zero*, an optional print-and-play group game. It offers a different experience of making decisions under pressure. The book's solo exercises provide a complete route if you never play.

Optional AI conversations

An assistant can explain a term, challenge a proposed control or quiz you. Try your own answer first, then ask it to identify an assumption you missed. Keep the evidence trail: a fluent explanation or invented incident is not a verified source. Three chapters include longer conversation prompts; other chapters use calculation, annotation and evidence checks.

Use fictional inputs from this book. Do not paste real phishing emails, credentials, session tokens, personal information, internal logs or production firewall rules into a public assistant. If you adapt an example, remove sensitive details and follow your organisation's approved data-handling rules. A generated configuration is a draft to test in an authorised environment, never a ready-to-deploy answer.

For more on this method, see *Conversation, Not Delegation* (https://michael-borck.github.io/conversation-not-delegation/). No companion book is required here.

The series trunk is *Conversation, Not Delegation* for method and *The Human Edge* for professional judgement. This book is the branch for defensive security.

Reading claims and sources

Examples labelled fictional are teaching inputs. Historical incidents and consequential technical claims have nearby sources, with a curated trail in Further Reading. Legal summaries describe selected regimes as checked in September 2026; applicability and facts matter. For an actual incident, involve the authorised technical and legal advisers.

This book is open access under the author's stated Creative Commons Attribution 4.0 International (CC BY 4.0 International) licence; original code examples use MIT. The free edition is the focus, and the series catalogue (https://books.borck.education) links the related books.

Acknowledgments

This book began with the students I have taught. Information security is usually met first as a pile of definitions: CIA, controls, threats, frameworks. It takes a while before those pieces click into a way of thinking. The moment they do, when someone stops memorising the list and starts asking "so where would this actually fail?", is the moment this book is written for. Their questions, and their confusion, shaped every chapter.

I am grateful to the colleagues who reviewed early drafts and pushed back where an explanation was still too clever or too vague. The book is clearer for it.

The open-source community behind Quarto, Pandoc, and GitHub made it possible to write, build, and publish this openly, with tools that are free and transparent.

How this book was written

This book was written with AI, using the method its companion volume describes. That companion, *Conversation, Not Delegation* (https://michael-borck.github.io/conversation-not-delegation/), argues that AI is most useful as a thinking partner you converse with, not a tool you hand your thinking to. I wrote *Assume Breach* that way. AI helped draft, question, and refine. It also produced its share of confident, plausible mistakes, and catching those was part of the work. That is the point the book makes about security as well: the machine produces a draft, and the human supplies the judgement

and takes responsibility. Every sentence here reflects the author's judgement. The AI made the work faster. It did not do the thinking.

Part I

Part I · Foundations

Chapter 1

Introduction to Information Security



What are we protecting, from whom, and how would we even know if we'd succeeded?

Begin with a system you can describe, then ask what a failure would harm. This chapter gives names to the protection goals and introduces the learning map used throughout the book. The first paper

task starts your dossier.

1.1 What this chapter covers

By the end you should be able to:

- Define information security and explain the **CIA triad**: confidentiality, integrity, availability.
- Explain when those three can conflict, and what "balancing CIA" means in practice.
- Say what a **security control** is, and what **defence in depth** buys you.
- Walk through the **phases of an attack**, and explain why a defender studies them.
- Distinguish **ethical** security work from malicious hacking, and see where the line sits.

1.2 Security is a property, not a product

You cannot buy security in a box. There is no single product, no switch, no certificate that makes a system "secure." Security is not a thing a system *has*. It is a property of how the whole system behaves, including the people using it. A bank vault with the door propped open is not secure no matter how thick the steel. A perfectly configured server whose administrator uses "password1" is not secure either. Security depends on how those choices interact. A weakness may expose one asset while other controls limit the damage.

That is why this book keeps returning to a defender's habit of mind rather than a checklist. Checklists go stale. The habit of asking "what am I protecting, from whom, and how would I know if it failed?" does not.

1.3 The CIA triad

Almost everything in information security can be traced back to protecting one of three properties of information. Together they are

called the **CIA triad** (no relation to the agency).

Confidentiality is keeping information from those who should not see it. When you think of "security," this is usually what comes to mind: secrets stay secret. Encryption, access controls, and passwords are all, at heart, confidentiality mechanisms. A confidentiality failure is a leak: data ending up in the wrong hands.

Integrity is keeping information correct and un-tampered. It is less obvious than confidentiality but often matters more. If an attacker cannot *read* your bank balance but can *change* it, you have a worse problem than a leak. Integrity mechanisms let you detect or prevent unauthorised change. A hospital needs to trust that a patient's recorded blood type is the one a doctor entered, not one an attacker altered.

Availability is keeping information and systems there when they are needed. A system nobody can reach is no more useful than one that was deleted. Availability is the property attacked by a denial-of-service, threatened by a **single point of failure**, and protected by backups, redundancy, and capacity planning. It is the property people forget until the day the system is down and the business stops.

1.3.1 Balancing CIA

The CIA properties can conflict, but they do not inevitably work against each other. A tested backup can improve both integrity and availability. Well-managed access can protect confidentiality while helping authorised users find the right information.

Trade-offs arise in particular designs. Extra approval steps may delay urgent access. More replicas improve availability but create more copies to protect. Preventing every change would preserve an old record while making a booking system unusable. Ask which harm matters, who is affected, and what evidence shows the control meets the need. A hospital's emergency-access arrangement can be different from a newspaper's embargo process without either abandoning security.

A fourth idea often sits alongside the triad: **non-repudiation**, the property that someone cannot credibly deny having done something. An audit trail and a digital signature can support a claim about an action. They still depend on trustworthy identity binding, key control and records. An account identifier alone does not prove which human acted or settle a dispute.

1.4 Controls and defence in depth

A **security control** is anything you put between an asset and a threat to reduce risk: a password, a firewall, a locked door, a policy, a training session, a backup. Controls come in familiar flavours: some **prevent** (a lock stops entry), some **detect** (an alarm notices it), and some **correct** (a backup restores what was lost). A good defence uses all three, because prevention eventually fails.

This is the idea the book is named for. **Defence in depth** means never trusting a single control to be enough. You assume each layer *will* fail, and you put another behind it. A castle did not rely on the outer wall alone. It had a moat, then a wall, then a gate, then an inner keep, so that breaching one defence still left the attacker facing the next. A modern system is the same: a firewall *and* patched software *and* least-privilege accounts *and* monitoring *and* backups. Layers reduce dependence on any one control. Shared credentials, a common administrator or a shared power supply can still defeat several layers together.

This is the practical face of **assume breach**. If you assume the attacker will get past your first control, you stop asking "is this wall strong enough?" and start asking "when this fails, what catches it?" That question builds better systems than the search for one perfect wall ever could.

1.5 The phases of an attack

To defend against attackers, you have to understand how they work. You are not learning this to imitate them. You study each step to

see what it *needs*, so you can take that away. Most intrusions move through recognisable phases:

1. **Reconnaissance.** The attacker gathers information about the target from the outside: who works there, what systems they run, what's exposed. Quiet, often legal, and easy to overlook.
2. **Scanning.** They probe the target directly for live systems, open ports, and running services, mapping the attack surface. In this book you can reason from supplied observations without probing any real system.
3. **Gaining access.** They exploit a weakness (a flaw, a weak password, a tricked user) to get a foothold.
4. **Maintaining access.** They entrench, so a reboot or a changed password doesn't lock them out.
5. **Clearing tracks.** They cover their trail, deleting logs and hiding activity to delay discovery.

Read that list as a *defender* and each phase becomes an opportunity. Reconnaissance can be made harder by exposing less. Scanning can be detected. Access can be denied with patching and strong authentication. Persistence and track-clearing can be spotted by monitoring that the attacker cannot fully erase. Understanding the attacker's process is where a defender's leverage comes from.

1.6 Ethical hacking, and the line

Much of what you'll learn in this book is, in mechanism, identical to what an attacker does: scanning systems, cracking passwords, capturing traffic. The difference is **permission and purpose.** An ethical practitioner acts with the owner's authorisation, within an agreed scope, to *find and fix* weaknesses before a real attacker exploits them. Unapproved testing may be unlawful and can harm others. Treat explicit permission and agreed scope as prerequisites, not optional courtesy.

This is where the old "hats" come from. A **white hat** works with authorisation, to find and fix. A **black hat** breaks in for their own gain or to cause harm. A **grey hat** acts without permission while

believing the intent is good. Good intentions do not move the line. What separates the three is authorisation.

That line is sharp and it matters. Practise only in an environment you own or are explicitly authorised to use. The skills are dual-use. Your professionalism is what makes them defensive.

> **Try it yourself**
>
> Begin with the fictional, paper-based Harbour case. Optional technical practice requires an explicitly authorised environment with checked boundaries. The practice appendix explains why a VM, NAT connection or container does not itself establish isolation.

1.7 A lifecycle to revisit

Use seven activities as a learning map: design, harden, detect, respond, recover, investigate, and audit/learn. People and decisions run through all seven. Activities overlap: evidence collection can begin during response, and a recovery test can reveal a design flaw before any breach occurs. This is the book's teaching map, not a mandated sequence. NIST's incident-response guidance (https://csrc.nist.gov/pubs/sp/800/61/r3/final) places incident work within continuing risk management.

1.8 Where this connects

The **CIA triad** and **defence in depth** are the lenses for the whole book. Almost every later control exists to protect one leg of the triad or to add one more layer. The next chapter begins the technical detail, looking at where software goes wrong and the malware that walks through the gaps.

1.9 Questions to consider

1. Give a real situation where **confidentiality** matters most, one where **integrity** matters most, and one where **availability** matters most. Use three different examples.
2. "More security is always better." Describe one case where tightening security actually made things *worse*. What was traded away?
3. Why would a *defender* bother learning the phases of an attack? What specifically do you gain by understanding how an attacker thinks?

1.10 Map an asset

Read Harbour's facts. In 10 minutes, choose one asset and write a four-column row: harm, plausible threat, control, and evidence that the control works. Then name one residual failure. A good row connects a specific harm to a test; "use encryption" without saying what it protects is not enough.

Case inputs and checks: Harbour dossier.

Chapter 2

Software Security & Malware



A software flaw can give an attacker a foothold. What happens next depends on the access and controls around it.

Software vulnerabilities, stolen credentials and deceptive messages can each give an attacker a foothold. This chapter concentrates on

software flaws and malware, then uses a historical breach to show how several failures can connect. Chapter 10 returns to the criminal market and the evidence left behind.

2.1 What this chapter covers

By the end you should be able to:

- Distinguish **design flaws** from **implementation flaws**, and explain **attack surface**.
- Explain what **CVE** and **CWE** are, and how **vulnerability disclosure** works (responsible disclosure, zero-day).
- Classify the main families of **malware**: viruses, worms, spyware, adware, trojans, RATs, ransomware, rootkits, and botnets.
- Explain how a **trojan** and a **botnet** actually work, and how we detect and prevent them.
- Trace how a real breach chains several of these together.

2.2 Where flaws come from

Software fails in two ways, and the distinction matters because they are fixed differently.

A **design flaw** is a mistake in the plan. The software does what it was designed to do, and the design itself is insecure. A system that emails you your own password in plain text has no bug. It works as intended, and the intention is the problem. Design flaws are expensive because fixing them means changing the plan, often after everything has been built on top of it.

An **implementation flaw** is a mistake in the building. The design was sound, but somewhere in the code a programmer made an error: mishandled a length, trusted an input they shouldn't have, forgot a check. Implementation flaws are one reason software needs patching. A patch may also change an insecure design; the categories help us reason but can overlap.

Every flaw is only a threat if an attacker can reach it. This brings us to **attack surface**: the sum of all the points where an attacker could try to get in. Every open port, every input field, every feature, every piece of installed software adds to it. A defender's instinct, seen already in the last chapter, is to *reduce* attack surface: turn off what you don't use, close what you don't need, and you shrink the number of doors an attacker can try. Attack surface is not fixed. It grows every time you add a feature, install software, or connect a new system, which is why security is never "done."

2.3 Naming the weaknesses: CVE and CWE

When there are millions of flaws across the world's software, you need a way to talk about them precisely. Two catalogues do this. A **CVE** (Common Vulnerabilities and Exposures) is a unique identifier for a vulnerability record, which can identify affected products and versions. A shared flaw may affect more than one product; the identifier gives people a common reference. A **CWE** (Common Weakness Enumeration) is one level up: a category of *type* of flaw ("improper input validation," "use after free") that recurs across many products. CVEs are the individual cases; CWEs are the patterns behind them. The CVE programme (https://www.cve.org/ResourcesSupport/FAQs) and CWE catalogue (https://cwe.mitre.org/) describe their different purposes.

2.3.1 Disclosure and the zero-day

When a researcher finds a flaw, **coordinated vulnerability disclosure** aims to get useful information to affected parties while reducing harm. A researcher reports through an authorised channel and coordinates with the vendor or a relevant response team. Publication timing must account for active exploitation, user protection and unresponsive vendors; it is not always "wait until patched". CERT/CC's disclosure guide (https://vuls.cert.org/confluence/display/CVD) explains the roles and trade-offs.

A **zero-day vulnerability** is commonly discussed as a flaw for which

defenders do not yet have an available fix, often before public disclosure. Distinguish the vulnerability from a zero-day exploit or attack that uses it. Terminology varies, so describe whether the vendor knew, whether a patch existed and whether exploitation was observed. Layered controls matter when patching cannot yet remove the exposure.

2.4 Malware: a field guide

Malware (malicious software) is the general term for any program written to do harm. The families are worth knowing because they spread and survive differently, and each difference is a place to defend.

2.4.1 Viruses and worms

A **virus** attaches itself to a legitimate file or program and spreads when that host is run and shared. It needs a carrier and, usually, a human to move it. A **macro virus** hides in the scripting built into documents (a spreadsheet or word-processor file), which is why "enable macros?" is a question worth treating with suspicion.

A **worm** is the more dangerous cousin: it spreads *on its own*, across the network, without needing a host file or a human to carry it. The 1988 Morris Worm, one of the first, showed the pattern that still holds: self-propagating code can move faster than any human response. For a defender, a worm's fuel is connectivity and unpatched systems, so segmentation and patching are what starve it.

2.4.2 Spyware, adware, and the willing click

Not all malware exploits a software flaw. **Spyware** collects information (keystrokes, browsing, credentials) and reports it back. **Adware** forces unwanted advertising and often tracks you to do it. A great deal of malware needs no vulnerability at all: it needs the user to *agree*, to click "install," to run the attachment, to grant the permission. This is the theme that returns in the human-factors chapter:

the most patched system in the world can still be undone by one willing click.

2.5 Trojans, RATs, and botnets

A **trojan horse** is malware disguised as something desirable: a game, a cracked application, a useful tool. The user runs it willingly, and while it may even do what it claims, it also does something else. A **Remote Access Trojan (RAT)** is a nasty variant: it hands the attacker remote control of the machine (files, webcam, keystrokes, the lot) as though they were sitting at it.

Scale this up and you get a **botnet**: a network of many compromised machines ("bots") under one attacker's control, taking orders from a command-and-control server. A botnet's owner can rent it out to send spam, mine cryptocurrency, or overwhelm a target with a distributed denial-of-service (DDoS) attack. The infamous Zeus family, for example, turned huge numbers of ordinary PCs into a credential-stealing network. The defender's insight is that a botnet's value is *aggregate*. No single infected machine matters much, which is why individual users underestimate the cost of being compromised.

Ransomware can encrypt data and disrupt services; attackers may also steal data and threaten to publish it. Tested, protected backups support recovery but cannot undo disclosure. Paying does not guarantee either restoration or deletion of stolen copies. Response therefore includes containment, assessment of exposed data and recovery, not just obtaining a decryption key. CISA's ransomware guide (https://www.cisa.gov/stopransomware/ransomware-guide) covers these connected risks.

2.6 Rootkits: hiding in the floor

A **rootkit** is malware that subverts a low level of the system (the operating system, or below it) so that it can hide everything above. By compromising the layer the rest of the system trusts to report

the truth, a rootkit can make its own files, processes, and network connections invisible to the tools you'd normally use to find them. This is why compromised machines are often rebuilt from scratch rather than "cleaned": once you can't trust the floor you're standing on, you can't trust anything it tells you.

2.7 Defending against malware

No single control stops all of this. That is why the book argues for layered defence. Signature-based detection recognises known patterns; other endpoint techniques look at behaviour or suspicious activity. Neither guarantees detection of an unfamiliar attack, so endpoint protection must be one layer among many. Patching closes the flaws worms and exploits need. Least privilege stops a compromised program reaching everything. Controls limit what software is allowed to run. User education targets the willing click. Monitoring catches the intrusion that gets through anyway. "We have antivirus" is not a security strategy. It is one thin layer, and a defender who relies on it alone has assumed the breach won't happen.

2.8 How it comes together: a real breach

Real attacks rarely use one flaw. They chain several. The 2013 Target breach is a classic worked example. Attackers first stole network credentials from a *third-party contractor* (a heating-and-cooling vendor with access to Target's systems). They used that foothold to reach the retail network and installed point-of-sale malware (the "BlackPOS" family) that scraped card data from checkout terminals. They exfiltrated tens of millions of card records while warning signs went unheeded. Read it as a defender and every stage is a control that could have caught it: tighter vendor access, network segmentation, restricted software execution, and someone acting on the alerts. Several controls offered possible opportunities to interrupt that chain. Which would have worked depends on implementation and timing. The 2014 Senate staff analysis (https://www.commerce.senate.gov/wp-content/uploads/me-

dia/doc/2014%200325%20Target%20Kill%20Chain%20Analysis.pdf) discusses possible missed opportunities using the public reporting then available; it is not proof of a unique counterfactual.

> Play it
>
> *Incident Zero*, a free, print-and-play security game, has a **Hardening** module where you spend a limited budget on layered defences and then a pentester attacks. It's the fastest way to *feel* why one control is never enough, and why "we have antivirus" is not a security strategy. incidentzero.retroverse.studio (https://incidentzero.retroverse.studio/)

2.9 Where this connects

The introduction laid out the phases of an attack, and this chapter fills in the "gaining access" phase. Stolen credentials, as in the Target breach, are the entry point the **authentication** chapter is about. Botnets and the criminal economy behind malware return in **cybercrime, botnets and forensics**.

2.10 Questions to consider

1. What's the difference between a **design flaw** and an **implementation flaw**? Give an everyday (non-software) analogy for each.
2. A worm and a trojan both spread, but they rely on different things. What does each one *need* to succeed, and which is easier to defend against?
3. Skim the Target breach. Write your one-sentence guess at the single change that would most likely have stopped it.

2.11 Trace a possible path

In 10 minutes, connect Harbour's shared administrator credential, export permissions and backup credentials into a possible attack

path. Label the path hypothetical. Name two places to interrupt it and one observation needed before claiming this was what happened. Do not invent a malware infection when the supplied evidence only records account activity.

Case inputs and checks: Harbour dossier.

Chapter 3

Authentication & Access



A successful login establishes an identity claim. The next question is what that account is allowed to do.

At Harbour, Rae needs to amend a booking but should not be able to export every participant's details. Checking who signs in and limiting what they can do are separate controls. This chapter follows both, including how systems protect password verifiers, recover accounts and end compromised sessions.

3.1 What this chapter covers

By the end you should be able to:

- Distinguish **identification**, **authentication**, and **authorisation**, and explain mutual authentication and ID proofing.
- Explain the **three factors** of authentication and what **multi-factor authentication** actually adds.
- Describe how a system **stores** credentials: cryptographic hashing, and why we add **salt**.
- Explain **Windows / NTLM** authentication at a high level and why it matters in practice.
- Analyse **password attacks** (dictionary, brute-force, rainbow tables) and the defences that beat each one: salting, lockout, length.

3.2 Three words people mix up

These three terms are used loosely in everyday speech and precisely in security, and the precision matters.

Identification is a *claim*: you state who you are (a username, an account number, a name). It is just an assertion, and on its own it proves nothing.

Authentication is the *proof* of that claim: you demonstrate that you are who you said. Typing the password that matches the username is authentication. This is the step attackers most want to defeat, because a successful fake here makes everything downstream trust them.

Authorisation is what happens *after*: given that we now believe who you are, what are you allowed to do? A verified user is not an all-powerful one. Authentication gets you through the door. Authorisation decides which rooms you may enter. Keeping these separate lets a system trust *who* you are while still limiting *what* you can do. This is the principle of least privilege in action.

Two refinements are worth naming. **Mutual authentication** is when *both* sides prove their identity. Not only does the server check you,

but you (or your software) check the server, so you're not handing your credentials to an impostor. **ID proofing** is the harder, real-world problem of establishing identity in the first place. How does a bank know the person opening an account is who they claim, *before* any password exists? Authentication verifies a claim you set up earlier. Proofing is how that claim was trusted to begin with.

3.3 The three factors

How can you prove who you are? Every method falls into one of three **factors**:

- **Something you know**: a password, a PIN, the answer to a question.
- **Something you have**: a phone, a hardware token, a smart card.
- **Something you are**: a fingerprint, a face, a voice (biometrics).

Passwords are widely used because they need little special hardware, but guessing, reuse and phishing make them risky on their own. Long, unique passwords generated and stored with a password manager reduce guessing and reuse. Arbitrary complexity rules can encourage predictable substitutions; rate limiting addresses repeated online guesses. Neither stops a user submitting a password to an impostor.

3.3.1 Multi-factor authentication

One important improvement is to require **more than one factor**: **multi-factor authentication (MFA)**. The power comes from *combining categories*: a password (know) plus a code from your phone (have) means stealing the password alone is no longer enough. The "have" factor takes many forms: a one-time code, a hardware **token** or key fob, or a **smart card** carrying a chip that proves possession. The "are" factor, biometrics, is convenient and hard to forget, but carries a permanent trade-off: you can change a breached password, but you cannot change your fingerprint. MFA is strong, but it is not magic. An attacker may still phish a one-time code or trick a user

into approving a prompt. The type of MFA matters. **Passkeys** and suitable security keys bind an authentication response to the legitimate service, resisting a fake login site in a way a relayed one-time code does not. Their enrolment, device protection and recovery paths still need care. A biometric often unlocks an authenticator locally rather than being sent to the website as a password. NIST's authenticator guidance (https://pages.nist.gov/800-63-4/sp800-63b/authenticators/) distinguishes these mechanisms.

3.4 What a system actually stores

Here is the idea that surprises most people, and it is the hinge of the whole chapter: **a well-built system does not store your password.** If it did, anyone who stole the database (or any administrator) would know every user's secret.

Instead, the system stores a **password verifier** produced by a purpose-built password-hashing scheme. At login it derives a verifier from the supplied password and stored parameters, then compares the result. A cryptographic hash is designed to resist inversion, but attackers can still guess inputs.

The scheme must make each guess expensive through a suitable work factor, and often memory cost. A fast general-purpose hash alone is not appropriate password storage. Libraries implementing schemes such as Argon2id handle these details; do not invent a scheme from hash primitives. OWASP's password-storage guidance (https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html) explains the choice and parameters.

3.4.1 Windows and NTLM

Windows' legacy **NTLM** authentication is useful optional background. It uses password-derived credential material in a challenge/response scheme. Stolen NTLM hash material may enable **pass-the-hash** authentication without recovering the plaintext password. Treat it as a credential, not as harmless scrambled data. It is

not a model for designing a new password store. Microsoft's NTLM overview (https://learn.microsoft.com/en-us/windows/win32/secauthn/microsoft-ntlm) describes the protocol, and NIST's threat guidance (https://pages.nist.gov/800-63-4/sp800-63b/security/) includes reuse of stolen authentication material.

3.5 Attacking passwords

If attackers can get the hashes but can't reverse them, how do they recover passwords? They don't reverse the hash. They **guess**, hash each guess, and look for a match.

A **dictionary attack** works through a list of likely passwords (common words, previously leaked passwords, predictable patterns) because real people rarely choose randomly. A **brute-force attack** tries every possible combination. For a finite candidate space it can exhaust every candidate, but the work may be infeasible. Length helps when it adds unpredictability; a long, well-known phrase can still be guessed early. The accelerator is a **rainbow table**: a giant precomputed lookup of hashes for enormous numbers of candidate passwords, so the attacker trades storage for speed and skips the recomputation.

3.5.1 The fix: salt

Rainbow tables have one fatal weakness, and the fix is elegant. Before hashing, the system adds a unique random value, a **salt**, to each password, and stores the salt alongside the hash. Now two users with the same password have *different* hashes, and the attacker's precomputed rainbow table, built without your specific salt, is worthless. Salting doesn't make any single password harder to *guess*. It prevents useful reuse of the same precomputed table across different salts. The attacker must work separately for each salt. The password-hashing scheme's cost settings, not the salt alone, make each guess expensive.

Distinguish **online** guessing against a login service from **offline** guessing against stolen verifiers. Login rate limits and lockouts

do not slow an attacker working on their own copy. Long unique passwords, proper verifier storage and protection of the database address that different threat.

3.6 The maths is rarely the weakest link

Everything above assumes the attacker has to *defeat* authentication. Often they go around it. A **keylogger** records the password as you type it, before any hashing happens. **Social engineering** just asks. A convincing email or phone call, and the user hands the secret over. **Sniffing** captures credentials sent in the clear across a network (which returns in the network-security chapter). And stealing the **password file** itself gives the attacker the hashes to work on at leisure. The recurring lesson of this book applies here: the cryptography is rarely what fails. The people and the plumbing around it are.

3.7 A word on single sign-on

One more idea you'll meet: **single sign-on (SSO)** lets you authenticate once and gain access to many systems, and **reduced sign-on** is a lighter version of the same convenience. Users love it, and it improves security in one way: fewer passwords means less reuse and fewer sticky notes. But it concentrates risk: a compromised identity provider or session may expose several services. SSO must still enforce per-service permissions, restricted administration and revocation; it need not give every account access to everything. Protect the identity provider and recovery process with appropriate MFA.

3.8 Permissions and recovery are part of the design

A receptionist may need to amend one booking without being allowed to export every participant's details. Enforce that rule on the

server for each action, not just by hiding a button. Review access when duties change or people leave.

Recovery is another entry path. A help desk that bypasses all checks when a caller sounds urgent can undermine strong login controls. Plan an independent identity check, recovery credentials where appropriate, and revocation of affected sessions. Changing a password alone does not prove that every existing session has ended.

3.9 Where this connects

The hashing this chapter relies on is the subject of the next chapter, **cryptography**. Sniffing for credentials reappears in **network security**, when we capture packets on the wire. And stolen credentials, as in the Target breach, are how many intrusions begin, a thread running back to **software security and malware**.

3.10 Questions to consider

1. If a website is breached, why is it so much better for you that they stored a **salted hash** of your password rather than the password itself?
2. MFA asks for a second factor. Give a realistic example where MFA still fails, where the attacker gets in anyway. What does that tell you about relying on it?
3. Why does adding **salt** defeat a rainbow table, when it doesn't make any single password harder to guess?

3.11 Separate login from permission

In 10 minutes, write the actions Mara, Rae and a volunteer should be allowed to perform at Harbour. Include one denied action for each role. Add a test for account recovery and session revocation. Success means a legitimate login does not silently confer bulk-export or administrator rights.

Case inputs and checks: Harbour dossier.

Chapter 4

Cryptography



A sound cipher should remain secure when its design is public and only the key is secret.

Cryptography can protect information sent through an untrusted network, but different operations solve different problems. Encrypting a booking, hashing an export and signing a message do not provide interchangeable assurances. You do not need to derive the algorithms here; you need to identify the protection, its assumptions

and the systems it depends on.

4.1 What this chapter covers

By the end you should be able to:

- Explain the difference between an **algorithm** and a **key**, and state **Kerckhoff's principle**.
- Distinguish **symmetric (one-key)** from **asymmetric (two-key)** cryptography and say what each is good for.
- Explain **hashing** vs **encryption**, and where digital signatures fit.
- Reason about attacks: brute-forcing small key spaces, known-plaintext and chosen-plaintext attacks.

4.2 The algorithm and the key

Every cipher has two parts, and telling them apart is the foundation of everything else. The **algorithm** is the method: the fixed procedure for scrambling and unscrambling. The **key** is the secret input that steers it: the particular setting that turns readable *plaintext* into scrambled *ciphertext* and back. The same algorithm with a different key produces different output. The set of all possible keys is the **key space**, and its size is one measure of a cipher's strength: the more keys an attacker would have to try, the better.

Here is where beginners' intuition usually goes wrong. It feels safer to keep the *algorithm* secret too. Surely hiding the method helps? **Kerckhoff's principle** says the opposite, and it is one of the most important ideas in the whole field:

> A cryptosystem should be secure even if everything about it, except the key, is public knowledge.

In other words, the security must rest **entirely on the secrecy of the key**, never on the secrecy of the algorithm. This sounds counter-intuitive until you see why it's a *strength*. Keeping a design secret can limit independent review without demonstrating that the

design is sound. If secrecy of the algorithm is the only protection, disclosure removes it. A public algorithm, published and attacked by the world's cryptographers for years, has earned its trust the hard way. This is why every cipher you should trust (the ones protecting your web traffic, your messages, your disk) has an openly published algorithm and keeps only the key secret. "Secret sauce" cryptography is a warning sign.

4.3 Symmetric cryptography: one shared key

The oldest and simplest arrangement is **symmetric** (or one-key) cryptography: the *same* key both encrypts and decrypts. If you and I share a secret key, I encrypt with it, you decrypt with it, and anyone without it sees only noise. Symmetric ciphers are fast and efficient, which is why they do the heavy lifting of bulk encryption: encrypting a whole disk, a database, or a stream of network traffic.

Symmetric cryptography has one stubborn problem, and it's not the maths: **key distribution.** Before we can communicate securely, we both need the same secret key. But how do I get it to you *securely* in the first place? If we could already send secrets safely, we wouldn't need the encryption. For two people who have never met, sharing a key over an untrusted network is the problem the whole system was meant to solve. For centuries this was cryptography's central limitation.

4.4 Asymmetric cryptography: a key you can publish

Asymmetric cryptography uses a mathematically related public/private key pair. The public key can be shared; the private key must be protected. Different schemes perform different jobs. Do not assume every pair can encrypt, decrypt and sign interchangeably.

With a public-key encryption scheme, a sender encrypts to the recipient's public key and the recipient decrypts using the private key. But the sender must establish that this really is the recipient's key. Pub-

lishing a key does not by itself solve the problem of authenticating it.

A **digital signature** is a separate operation: a signing algorithm uses a private key, and a verification algorithm uses the corresponding public key. Signing is not generally "encrypting with the private key". Verification checks the signature against the message. A valid signature supports a claim about the signing key, subject to trustworthy key identity and control; it does not by itself prove a human's intent. NIST's Digital Signature Standard (https://csrc.nist.gov/pubs/fips/186-5/final) describes these distinct signing and verification operations.

Real systems often combine public-key techniques with fast symmetric protection. In a certificate-authenticated TLS connection, the browser checks the server's certificate and name, the parties establish traffic keys, and symmetric authenticated encryption protects the traffic. Modern key agreement can derive a shared secret rather than sending a symmetric key encrypted with the server's public key. TLS 1.3 (https://www.rfc-editor.org/rfc/rfc8446.html) separates these steps. If certificate validation fails, do not bypass the warning just to continue.

4.5 Hashing: one-way, and not encryption

A **hash function** is often grouped with cryptography but does something different, and the difference matters. Encryption is *reversible*. With the right key, decryption recovers the original. A cryptographic hash is deliberately **one-way**: it turns any input into a fixed-length fingerprint that cannot be reversed to recover the input. You met this in the last chapter, where systems store the hash of a password rather than the password itself.

Encryption aims to protect confidentiality. A cryptographic hash gives a compact comparison value designed to make it infeasible to find a different input with the same digest. Collisions are mathematically possible; "every change must produce a different hash" is not the guarantee.

To check a downloaded file, compare its digest with an expected value obtained through a trusted channel. If an attacker replaces both the file and its advertised digest, comparison alone proves nothing about authenticity. A keyed message authentication code or a signature can add authentication under the relevant key assumptions.

Encryption alone also need not detect tampering. **Authenticated encryption** combines confidentiality with an integrity check, rejecting altered protected data. Use a maintained protocol or library rather than assembling your own combination. Key theft, a compromised endpoint or an authorised but malicious application session can still expose the plaintext.

4.6 Attacks, and the limits of weak crypto

Understanding how ciphers *fail* sharpens the intuition. The simplest failure is a **key space too small to matter.** The classic Caesar cipher shifts each letter by a fixed amount. But there are only twenty-five possible shifts, so an attacker tries them all in seconds. A **monoalphabetic substitution** cipher, where each letter maps to another, has an enormous key space yet still falls easily, because it leaks the *structure* of the language: letter frequencies survive the substitution, and ‘e’ is still the commonest letter. A big key space is necessary but not sufficient. A cipher can be broken by the patterns it fails to hide.

Cryptographers also reason about what an attacker gets to *see*. In a **known-plaintext attack**, the attacker has some plaintext *and* its matching ciphertext and uses the pair to attack the key. In the stronger **chosen-plaintext attack**, they can feed in plaintext of their choosing and study the output. A cipher worth trusting must withstand even an adversary with those advantages. That is why real ciphers are published and battered by experts rather than kept secret and hoped over.

Well-chosen modern cryptography can make direct cryptanalytic attacks impractical. Attackers may instead exploit the surrounding

system. They steal the key from a poorly protected file, trick a user into revealing it, exploit a bug in the *implementation* of a sound algorithm, or take the data before it's encrypted or after it's decrypted. Even ransomware, which weaponises encryption *against* you, succeeds through the plumbing (how it gets in and what it can reach) rather than by breaking the cipher. The defender must examine both the cryptographic choice and its operating conditions; neither should be assumed correct merely because encryption is present.

4.7 Where this connects

Password verification uses the hashing ideas introduced in **authentication**. The encrypted tunnel a VPN builds rests on the symmetric, asymmetric, and key-exchange ideas here, as **VPN and firewalls** will show. And the secure connection behind every reputable website is the hybrid scheme described above, which returns in **web security and data protection**.

4.8 Questions to consider

1. Why is it a *good* thing that strong ciphers publish their algorithms openly?
2. Symmetric crypto is faster; asymmetric solves key distribution. Sketch how a real system might use *both*.
3. What's the practical difference between **hashing** a message and **encrypting** it? When would you want each?

4.9 Choose the protection

In 10 minutes, decide what protects Harbour's traffic, stored records and backup comparison values. Explain one limitation of each choice. Your answer should recognise that an application can export plaintext after login, and that a hash beside an editable file needs a separately trusted reference. Compare with the worked dossier only after making your choices.

Case inputs and checks: Harbour dossier.

Part II

Part II · Managing Risk

Chapter 5

Risk Management



You can't protect everything equally, so you don't. Risk management is how a defender decides where the money and effort go.

Everything before this chapter was about *how* to defend: controls, cryptography, layered defence. This chapter is about *where to spend the effort*. It is the moment security stops being a technical subject and becomes a business one. No organisation has unlimited money,

time, or patience, so a defender who tries to protect everything equally protects nothing well. Risk management is the discipline of deciding, honestly and defensibly, what to protect, how much to spend, and what to leave alone. It is also where "assume breach" turns into a budget.

5.1 What this chapter covers

By the end you should be able to:

- Define **risk**, **risk appetite**, and explain why appetite varies between organisations.
- Explain **cost-benefit analysis** and the difference between **intrinsic** and **acquired** value.
- Calculate **single loss expectancy (SLE)** and **annual loss expectancy (ALE)**, and use them to judge whether a control is worth it.
- Describe the main **risk treatments**: avoid, mitigate, transfer, accept.

5.2 What risk actually is

In everyday speech "risk" means danger. In security it means something more precise and useful: risk is the combination of **how likely** something bad is and **how much it would hurt** if it happened. A useful shorthand is $\textit{risk} = \textit{likelihood} \times \textit{impact}$. Neither alone tells you much. A catastrophic event that almost never happens may be a smaller risk than a minor one that happens weekly. Treating them as equal, as a fear-driven approach does, wastes effort on the dramatic and ignores the routine.

Because risk is about likelihood and impact, it is always relative to a particular organisation. That is where **risk appetite** comes in: how much risk a given organisation is willing to accept in pursuit of its goals. A hospital and a social-media start-up face different threats, value different things, and answer to different regulators, so they will rationally accept different risks. Risk appetite is a deliberate

statement of how much uncertainty an organisation will live with. It is neither recklessness nor timidity. Two well-run companies facing the same threat can make opposite, equally defensible decisions. They should be able to explain the constraints behind those choices.

5.3 What are you protecting, and what's it worth?

You cannot judge a risk without knowing the value of what's at stake, and value is subtler than a price tag. An asset has **intrinsic value** (what it is worth in itself, like the replacement cost of a server) and **acquired value** (the worth it takes on from its role and context). A customer database might contain a few dollars of storage, but the acquired value (the trust, the regulatory exposure, the competitive harm if it leaked) can be enormous. Defenders who price only the intrinsic value under-protect the things that matter most, because the damage from losing information usually dwarfs the cost of the hardware holding it.

Once you can value what's at stake, you can do **cost-benefit analysis**: weigh what a control *costs* against the loss it *prevents*. This provides one input to a security decision. It has an uncomfortable implication people resist: sometimes the right answer is to *not* buy the control, because it costs more than the harm it prevents. That judgement is constrained by legal duties, safety and the organisation's stated limits. A positive financial return is not the only reason to protect people.

5.4 Putting numbers on it: SLE, ARO, ALE

To make cost-benefit concrete, security borrows a simple model from insurance. Three quantities do the work. **Single Loss Expectancy (SLE)** is how much one occurrence of the event would cost you. **Annualised Rate of Occurrence (ARO)** is how many times a year you expect it to happen, and it can be a fraction: 0.2 means once every five years. **Annual Loss Expectancy (ALE)** is the expected

cost *per year*, `SLE × ARO`.

Here is a worked example. Suppose a breach of your customer database would cost about **$500,000** to clean up, notify, and recover from. That is the SLE. You judge such a breach might happen once every five years, so the ARO is **0.2**. The annual loss expectancy is:

```
ALE = SLE × ARO = \$500,000 × 0.2 = \$100,000 per year.
```

That $100,000 is what this risk "costs" you in expectation each year, and it's the number a control has to beat.

5.5 Is the control worth it?

Now you can judge a control honestly. Say better monitoring and encryption cost **$40,000 a year** and would cut the breach rate to once every twenty years (ARO 0.05). The new annual loss expectancy becomes `\$500,000 × 0.05 = \$25,000`. So the control *reduces* your ALE from $100,000 to $25,000, a benefit of **$75,000 a year**, for a cost of $40,000. It saves you $35,000 net, so it is worth buying.

For this financial model, **a control has positive expected net benefit when the reduction in ALE exceeds its annual cost.** That is not a complete acceptance rule for safety, privacy or regulatory obligations. Turn the numbers around and the answer flips. A $50,000 control that only reduces an ALE from $30,000 to $5,000 *loses* you money, and buying it out of a vague sense that "more security is better" is the mistake this chapter exists to prevent.

Two honest cautions come with the arithmetic. First, notice that even after the good control, $25,000 of annual risk remains: this is **residual risk**, the risk left over after treatment. You almost never reduce risk to zero, and pretending otherwise is how organisations get surprised. Second, these numbers are *estimates*: SLE and ARO are educated guesses, and some impacts (a ruined reputation, harm to people, loss of life) resist being priced at all. Treat quantitative risk analysis as a discipline for thinking clearly. The numbers structure the judgement; they do not replace it.

5.6 The four things you can do with a risk

Once a risk is identified and sized, you have four options. They are the vocabulary every security professional shares:

- **Avoid.** Don't do the risky thing at all. Stop offering the feature, don't collect the data, close the exposure. This can remove the particular exposure, but may introduce other risks. It is often impractical because the activity is part of the business.
- **Mitigate.** Reduce the likelihood or the impact with controls. This is most of what the rest of this book is about: patch, encrypt, segment, train, monitor. You rarely eliminate the risk; you shrink it to an acceptable size.
- **Transfer.** Share specified financial consequences through **insurance** or a contract. Coverage, exclusions and provider obligations matter. Outsourcing does not automatically transfer your responsibility for customers or legal compliance.
- **Accept.** Decide the risk is small enough, or the fix too costly, to live with it. This is a legitimate, deliberate choice, *provided* it's made consciously and by someone with the authority to make it. Record the owner, reasoning, residual exposure and review date. An unexamined risk is not the same as deliberate acceptance.

Organisations use resources with different purposes: the **ASD Essential Eight** sets out mitigation strategies and maturity expectations; **ISO/IEC 27001** specifies an information-security management system; the **NIST Cybersecurity Framework** organises cybersecurity outcomes. They are not interchangeable pass/fail checklists. The NIST CSF (https://www.nist.gov/cyberframework) provides a starting point for understanding outcomes and governance. Use the framework to structure evidence and responsibilities, then explain how the chosen treatment addresses the organisation's actual exposure.

Play it

Incident Zero's **Audit & Compliance** module has you score an organisation's controls pass/fail against a real framework, and then defend your judgement to the table. It turns the abstract choice between *accept, mitigate, transfer, avoid* into a call you have to argue for out loud. incidentzero.retroverse.studio (https://incidentzero.retroverse.studio/)

5.7 Where this connects

The next chapter, **incident and disaster planning**, is what you do when a risk you accepted or mitigated happens anyway. Every control elsewhere in the book is a *mitigation*, and this chapter is how you decide which ones are worth deploying. A maturity assessment against a standard like the Essential Eight is this same risk-based reasoning, applied control by control.

5.8 Questions to consider

1. Two companies face the same threat but treat it differently. One buys insurance, one consciously accepts it. How can both be *rational*?
2. A control costs $50k/year and reduces an ALE of $30k/year to $5k/year. Should they buy it? Show your reasoning.
3. Give a real risk your own organisation (or somewhere you've studied or worked) probably **accepts** rather than fixes. Why is that a defensible choice?

5.9 Calculate, then challenge

In 15 minutes, calculate Harbour's expected-loss reduction and net benefit for option A. Repeat for post-control annual rates 0.02 and 0.09. Then choose an affordable set of controls and record what it leaves unresolved. Success requires correct arithmetic, stated

assumptions and a named risk owner; a positive estimate alone is not approval to ignore binding duties.

Case inputs and checks: Harbour dossier.

5.10 Take it to the AI

Optional: attempt the paper task first. Use only the fictional case, never real messages, credentials, personal information or internal logs. Check any external claims against the primary sources; do not deploy generated rules.

> Act as a reviewer of my fictional Harbour risk decision. Identify the assumption most likely to change my conclusion. Use only the printed costs and estimates; do not invent product effectiveness. Ask one question, wait for my answer, then explain how I could obtain better evidence.

Chapter 6

Incident & Disaster Planning



The question isn't whether something will go wrong, but whether you're ready when it does.

At Harbour, an unexpected login prompt and a bulk-export event arrive within minutes of each other. Who can suspend access, preserve the logs, talk to the service provider and decide what the centre tells

participants? This chapter turns those questions into a response plan. Preparation and recovery tests belong before the first incident.

6.1 What this chapter covers

By the end you should be able to:

- Distinguish **incident response**, **business continuity**, and **disaster recovery**.
- Explain **RTO** and **RPO** and why they drive the plan.
- Describe what a good **incident response** process looks like (detect respond recover learn).
- Explain the role of **backups**, alternate sites, and testing the plan.

6.2 When prevention runs out

Patching and access controls lower the chance of compromise. Detection, response and recovery reduce harm when prevention fails. They also help with mistakes and outages unrelated to an attacker.

Preparation means deciding authority, keeping contact information available, and testing recovery while there is time to correct a failure. A response plan should work when the usual administrator is absent and the primary communication system is unavailable.

6.3 Incident response: the lifecycle

An **incident** involves an actual or imminent threat to security. A suspicious event needs triage; staff should not wait for certainty before reporting it. A response process turns incomplete observations into proportionate action:

- **Detect and assess.** Record the observation, time and affected service. Decide urgency and scope, distinguishing facts from assumptions.

- **Respond.** Contain damage under assigned authority. Preserve useful evidence where feasible while restricting compromised access. Record actions that change or destroy evidence and why they were necessary.
- **Recover.** Address the cause and persistence, restore or rebuild from an acceptable state, and validate access and integrity before reconnecting. A successful restore is not proof the attacker has been removed.
- **Learn.** Review controls, decisions and communication. Assign improvements and check that they are completed.

These activities overlap. Evidence collection and analysis begin during response, not only after restoration. Do not indiscriminately wipe a system before consulting the response lead; equally, do not leave serious harm continuing solely to obtain perfect evidence. NIST SP 800-61 Rev. 3 (https://csrc.nist.gov/pubs/sp/800/61/r3/final) places incident response within continuing risk management.

Two things make or break response in practice, and neither is technical. **Roles** must be decided in advance (who declares an incident, who talks to whom, who has the authority to take a critical system offline), because a crisis is the worst possible time to be working out the org chart. And **communication** must be planned: internal coordination, and the harder question of what you tell customers, regulators, and the public, and when.

6.4 Business continuity and disaster recovery

Two related disciplines sit alongside incident response, and people constantly confuse them.

Business continuity is the big picture: keeping the *organisation* running during a serious disruption of any kind, not just a cyberattack. If the office floods, the payroll system dies, or ransomware locks every file, how does the business keep serving customers and paying staff? It covers people, processes, premises, and priorities, the whole organisation's ability to function through a crisis.

Disaster recovery is the technical subset of that: restoring the

IT systems and data after they've been lost or damaged. Business continuity asks "how does the company keep operating?"; disaster recovery asks "how do we get the systems and data back?" Recovery serves continuity (you restore the systems so the business can continue), but they are not the same plan, and an organisation needs both.

6.5 The two numbers that drive the plan: RTO and RPO

Disaster recovery turns on two deceptively simple targets, and getting them straight is the most useful thing in this chapter.

Recovery Time Objective (RTO) is the target elapsed time for restoring an agreed service level after disruption. Set it within the organisation's maximum tolerable disruption, allowing for dependencies and validation. A trading platform's RTO might be minutes; an internal wiki's might be days.

Recovery Point Objective (RPO) is *how much data you can afford to lose*: the maximum tolerable gap between your last good backup and the moment of failure. An RPO of one hour means you can lose at most an hour's work; an RPO of a day means losing a day's is acceptable.

The RPO constrains your recovery design, including backup or replication frequency and the availability of usable recovery points. This is where plans contradict themselves. If a company declares an RPO of one hour but backs up only once a night, the plan is broken on its face: a failure just before the nightly backup could lose nearly twenty-four hours of data, not one. The stated goal and the actual practice don't match, and no one notices until the day it matters. Test the gap using a failure time near the end of the backup interval. A copied but corrupt or compromised backup does not meet the target. NIST's contingency-planning guidance (https://www.nist.gov/privacy-framework/nist-sp-800-34) helps distinguish recovery objectives from tolerable disruption.

6.6 Backups and alternate sites

Backups support recovery. The **3-2-1** rule of thumb means three copies, on two types of media, with one off-site. Geographic separation protects against a local disaster. **Offline or appropriately immutable protection** addresses a different problem: an attacker modifying or deleting backups.

An off-site copy reachable with the same compromised administrator account may still be lost. Separate administration, retention controls and tested restoration matter. Check that recovery documentation and required keys remain available during an outage. Neither a backup job's success message nor a provider's promise substitutes for an observed restore test.

Good backups can reduce pressure to pay for decryption. They do not erase data already stolen, resolve legal obligations or guarantee a clean recovery. CISA's ransomware guide (https://www.cisa.gov/stopransomware/ransomware-guide) covers prevention, data theft and recovery together.

For systems that can't tolerate much downtime, recovery relies on an **alternate site**, a second location ready to take over. These come in flavours by cost and readiness: a **hot site** is fully equipped and running, ready in minutes; a **cold site** is just space and power you'd have to build out, cheap but slow; a **warm site** sits in between. Which you choose is a risk-and-cost decision: the cost-benefit reasoning of the previous chapter, applied to recovery.

6.7 The clock and the crowd

A response plan needs separate owners for technical recovery, communication and legal assessment. Do not promise that "every breach has a 72-hour clock". Under GDPR, the controller's regulator-notification deadline generally runs from awareness of a personal-data breach, with a risk-based exception; communication to affected people has a different threshold. Australia's NDB scheme uses its own coverage, serious-harm and assessment rules. Web Security

and Data Protection sets out the distinction and primary sources.

Record when relevant facts became known and escalate promptly to the responsible adviser. Investigation may continue while notification duties are addressed. Customers need accurate information about what is known, what is uncertain and what they should do.

A ransom demand adds pressure. Payment does not guarantee restored data or deletion of stolen copies, and legal restrictions may apply. The authorised decision-makers need technical and legal advice for the actual circumstances. This book's exercises do not ask the reader to negotiate or decide a real ransom payment.

6.8 Testing the plan

One rule outweighs all the rest: **an untested plan is only a hope.** Recovery plans fail at the moment they're needed for depressingly ordinary reasons: the backups were running but never restored-tested and turned out to be corrupt; the recovery documentation was itself on the system that went down; the one person who knew the process had left. Every one of these is caught by *rehearsing*: restoring from backups for real, running tabletop exercises, walking the team through the crisis while it's still hypothetical. The organisations that recover smoothly are the ones that practised; the ones that discover their plan's flaws during the real disaster paid the worst possible price for the lesson.

> Play it
>
> *Incident Zero* has two modules that live in this chapter. **Incident Response** races you to uncover a hidden attack chain before time and budget run out; **Disaster Recovery** drops you into a live breach with a 72-hour notification clock and the ransom decision on the table. Reading about incident response is one thing; running one against a clock is another. incidentzero.retroverse.studio (https://incidentzero.retroverse.studio/)

6.9 Where this connects

This chapter is what you do when a risk from **risk management** happens anyway, and the hot, warm, and cold-site choice is that chapter's cost-benefit reasoning applied. Investigation starts alongside containment; its evidence and attribution methods are taken up in **cybercrime, botnets and forensics**. And the breach-notification obligations here are grounded in the data-protection law of **web security and data protection**.

6.10 Questions to consider

1. What's the difference between **business continuity** and **disaster recovery**? Give an example of each.
2. A company backs up nightly. Their RPO is "no more than one hour of data lost." What's wrong, and what would you change?
3. Why do so many disaster-recovery plans fail at the moment they're needed? What single practice would most reduce that?

6.11 Rehearse the first decisions

Use Harbour's evidence cards for a 15-minute paper tabletop. Decide who acts at 09:15, 09:20 and 09:25; distinguish reporting from attribution. Calculate the rehearsal's recovery time and both printed recovery-point gaps. State what still needs checking before reconnecting. Success requires preservation alongside containment and separate treatment of exported data.

Case inputs and checks: Harbour dossier.

Part III

Part III · Defending Systems & Networks

Chapter 7

Network Security



What an observer can learn from traffic depends on where they can see it and how it is protected.

Harbour's booking service depends on several network paths: customers reach the web application, staff use a VPN, and the application queries a database. Some traffic crosses infrastructure the centre does not control. An observer on a path may learn from traffic without compromising either endpoint. This chapter explains

those observation limits and how to protect a connection.

7.1 What this chapter covers

By the end you should be able to:

- Explain how **scanning** and **sniffing** work and what they reveal.
- Describe what a **packet capture** shows, and why unencrypted traffic is so revealing.
- Describe common network attacks (**spoofing**, **man-in-the-middle**) and their defences.
- Explain why **encryption in transit** matters, and what segmentation and monitoring add.

7.2 A small network primer

A **host** is a device or system connected to a network. Its **IP address** helps route packets to it; a **port** helps identify a service on that host. A service is a listening application, such as a web server. **DNS** maps names to addresses. A switch connects devices locally; a router moves traffic between networks. A **subnet** groups addresses for routing purposes.

A **trust boundary** is where rules about access or trust change, such as between guest Wi-Fi and staff systems. A line on a drawing does not enforce that boundary: routing, filtering and access controls must implement it.

7.3 The network as an attack surface

Recall the phases of an attack from the first chapter: reconnaissance, then scanning, then gaining access. Networks are where the first two of those live. Before an attacker can exploit a system, they have to *find* it and learn what it's running. A network, by its nature, answers questions. Ask a machine "are you there?" and, unless told otherwise, it politely replies. Ask "what services are you running?"

and its open ports announce them. Each reachable service adds an exposure to examine. Defaults vary, and a lack of response does not prove a device or service is absent.

7.4 Scanning: mapping the target

Scanning is how an attacker (or a defender auditing their own network) turns a vague target into a map. It usually happens in layers. First, find the live hosts: a sweep that asks every address on a network "are you there?" and notes who answers. Then, for each live host, find the open **ports**. A port is a numbered doorway, and each open one usually corresponds to a running **service** (a web server, a file share, a remote-login service). Finally, probe those services to learn *which software and version* is behind them, because a known version can be matched against known vulnerabilities from the software-security chapter.

The result is a blueprint: here are your machines, here are the doors that are open, here's the software behind each one. To an attacker, that's a shopping list. To a defender, the same scan is an audit. That is why the tools are dual-use, and why the defensive instinct is to *close the doors you don't need*. Every service you're not using is a door you can shut, shrinking the blueprint an attacker gets to work from.

7.5 Sniffing: reading the wire

Scanning finds the doors. **Sniffing** reads what goes through them. A network interface normally ignores traffic not addressed to it, but put it into a listening ("promiscuous") mode and it will capture *everything* it can hear: a **packet capture** of the raw conversations crossing the wire. A tool can decode supported protocols from the captured packets.

The phrase *it can hear* matters. On a switched network, another host's unicast traffic may never reach your capture interface. Promiscuous mode does not make the switch send it there. Wi-Fi cap-

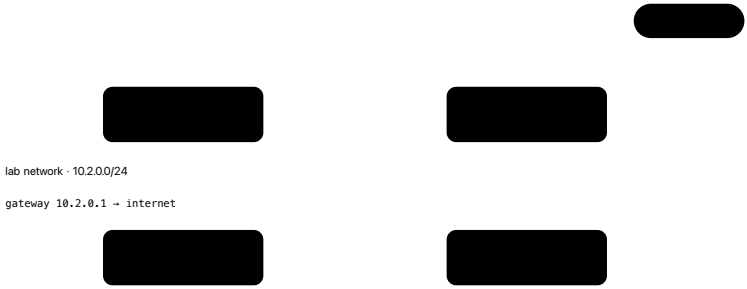


Figure 7.1: A small network as a scanner sees it: an attacker host and three servers, each answering with the ports it has open. This illustrative network maps web, app and file services across `10.2.0.0/24`; no companion lab is required.

ture also depends on the adapter, access-point settings and link protection. Explain the observation point before claiming what a capture reveals. Wireshark's Ethernet guidance (https://wiki.wireshark.org/capturesetup/Ethernet) sets out these limits.

One thing lands hardest the first time someone sees it: **unencrypted application traffic can expose its contents to a listener at a suitable capture point.** Older protocols were designed in a more trusting era and send everything unencrypted: the web's original HTTP, file transfer with FTP, remote login with Telnet, some email protocols. Capture that traffic and you don't just see *that* a conversation happened. You see its contents: the pages requested, the files moved, and the **usernames and passwords typed in the clear**. This is the sniffing that the authentication chapter warned about, made concrete: the strongest password in the world is worthless if it crosses the network in plain text for anyone listening to read. It is unsettling to witness, and it converts "you should use encryption" from advice into an obvious necessity.

7.6 Attacks in the middle

Listening is passive. Network attacks get more dangerous when the attacker stops observing and starts *interfering*.

Spoofing is forging an identity on the network: sending traffic that claims to come from an address that isn't yours. An attacker might spoof a trusted machine's address to slip past a filter, or forge the source of an attack to hide their tracks. The network's willingness to believe what packets *claim* about themselves is the weakness being exploited.

The more powerful move is the **man-in-the-middle** (or on-path) attack: the attacker positions themselves *between* two parties, so that everything each side sends passes through them first. If the protocol lacks effective authenticated protection, they may read or alter the conversation: changing a bank account number in transit, injecting malicious content, or capturing credentials as they flow past. Two common techniques get an attacker into that position on a local network. **ARP spoofing** tricks nearby machines into routing their traffic through the attacker by lying about which hardware address belongs to which machine. **DNS spoofing** answers a name lookup with the wrong address, so the victim connects to the attacker's server while believing it's the real one. The details matter less than the shape: the attacker exploits the small, trusting mechanisms that make a network convenient, and turns them into a wiretap.

7.7 Why the local attacker is special

Notice that ARP spoofing and casual sniffing both require the attacker to be *on the same network*. This is why an attacker who has gained a foothold on your local network, or who is sharing the coffee-shop Wi-Fi, can do things a purely remote attacker cannot. They can hear the local chatter, impersonate local machines, and slip into the middle of local conversations in ways that someone out on the wider internet, several routers away, cannot manage. It's a concrete reason the *inside* of a network can never be treated as automatically safe. This is the assume-breach mindset applied to

your own LAN.

7.8 Defences: making the wire private again

The good news is that the central weakness, traffic readable by anyone listening, has a decisive fix, and you already met it.

Encryption in transit is the answer to sniffing. When traffic is encrypted (the web's HTTPS, which is HTTP wrapped in the TLS scheme from the cryptography chapter), an eavesdropper still captures the packets but sees only unreadable ciphertext. The credentials, the page contents, the files: all gibberish to anyone without the key. It protects the encrypted contents, but packet timing, sizes and some addressing information remain observable. Protocol and configuration choices determine what else is visible. TLS must also authenticate the intended endpoint to resist an active impostor; ignoring certificate errors undermines that check. That is why the migration of the web from HTTP to HTTPS was one of the most important security improvements of the last decade, and why sending anything sensitive over an unencrypted protocol is now indefensible.

Two more layers back it up. **Segmentation** divides a network into zones with controlled routes between them, so that a foothold in one part doesn't grant free movement across the whole. It's the same idea that starves a worm (from the malware chapter) and contains an incident (from the planning chapter). **Monitoring** watches network traffic for the tell-tale signs of trouble (a host scanning its neighbours, traffic to a known-bad address, a volume spike), so that the intrusion you couldn't prevent is at least *detected*, feeding straight into the incident response of the previous chapter. Encryption hides the contents, segmentation limits the movement, and monitoring catches what gets through. No single one is enough.

> Play it
>
> *Incident Zero*'s **Network Building** module makes you buy an architecture you can't fully afford, and the gaps you leave come

back to bite in later rounds. It's a quick, concrete lesson in why security is a budget problem as much as a technical one. incidentzero.retroverse.studio (https://incidentzero.retroverse.studio/)

7.9 Where this connects

The credentials a sniffer harvests are the ones **authentication** taught you to protect, and encryption in transit is what stops them leaking. That encryption is the hybrid TLS scheme from **cryptography**, in action. And the next chapter, **VPN and firewalls**, covers the two defences that build on this one: a firewall to control what traffic is allowed, and a VPN to protect traffic within its configured tunnel.

7.10 Questions to consider

1. Name one everyday thing you do online that would be **visible in plaintext** to someone sniffing the network, and one that wouldn't. What's the difference between them?
2. Why can an attacker on the *same* network do things a purely remote attacker can't?
3. Imagine you captured your own device's traffic for one minute. What do you think you'd be able to read, and what would be hidden? What does that difference tell you about how you should send sensitive information?

7.11 State what an observer can see

In 10 minutes, draw Harbour's public, staff, guest and database boundaries. For a passive observer at the guest access point, distinguish encrypted content from routing/timing information and unknown visibility. Do not assume the observer receives every staff packet. Name a boundary test you would request from an authorised administrator; do not run it yourself.

Case inputs and checks: Harbour dossier.

Chapter 8

VPN & Firewalls



A firewall decides what traffic to let through. Writing a good rule is harder than it sounds, and that difficulty is the lesson.

The last chapter left us with two problems: networks let attackers *reach* systems they shouldn't, and they let attackers *read* traffic they shouldn't. This chapter is about the two classic defences that answer those problems. A **firewall** controls what traffic is allowed to pass.

It decides who gets to reach what. A **VPN** protects traffic in transit. It makes the readable wire private again. Both are everyday tools, and both are widely misunderstood. Understanding what each does *and does not* do is worth more than knowing how to configure either.

8.1 What this chapter covers

By the end you should be able to:

- Explain what a **firewall** does and the difference between packet-filtering, stateful, and application-layer firewalls.
- Reason about **firewall rules**: default-deny vs default-allow, and why order matters.
- Explain what a **VPN** provides (confidentiality and integrity over an untrusted network) and what it does *not*.
- Describe **defence in depth** in a network context: no single control trusted alone.

8.2 What a firewall does

A **firewall** is a checkpoint. It sits at a boundary between networks (classically between a trusted internal network and the untrusted internet) and inspects the traffic trying to cross, allowing what's permitted and blocking the rest. That's the whole idea: a gate with a guard who checks each arrival against a set of rules. A firewall does not try to block everything. Firewalls run at network boundaries, on individual hosts, and increasingly in cloud environments, but the job is always the same: decide, packet by packet or connection by connection, what's allowed through.

The difficulty, and the reason this chapter exists, is that the guard is only as good as the rules they're given. A firewall doesn't understand *intent*; it applies your policy literally. So the security a firewall provides is the quality of the thinking that went into its rules. It's a recurring truth in this book: the tool is easy, the judgement is hard.

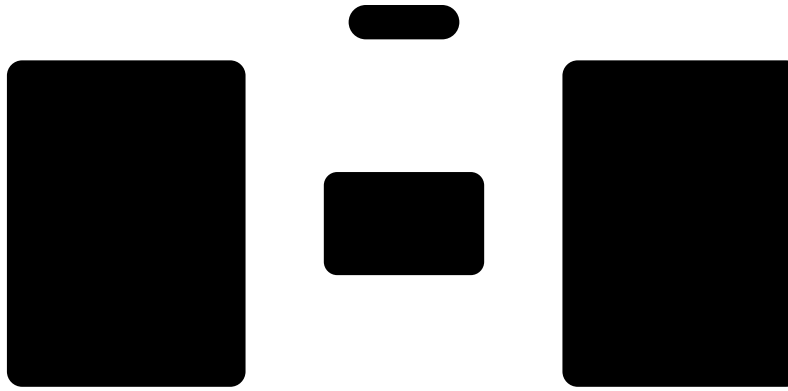


Figure 8.1: A firewall between two zones: an *inside* network it protects and an *outside* it distrusts. Everything crossing between `10.1.1.0/24` and `10.1.2.0/24` passes through the firewall's routes and filters. This is an illustrative topology, not a required companion lab.

8.3 The three kinds of firewall

Firewalls differ in how deeply they look at traffic, and the progression is worth knowing because each level trades speed for insight.

A **packet-filtering** firewall is the simplest: it examines each packet's headers in isolation (source and destination address, port, protocol) and permits or blocks based on those alone. It's fast, but it has no memory. Because it looks at each packet on its own, it can't tell whether a packet is a legitimate reply to a request you made or an unsolicited probe from an attacker; both look the same in isolation.

A **stateful** firewall fixes that by tracking the *state* of connections. It remembers that you opened a connection to a web server, so it knows to allow that server's replies back in, while still blocking unsolicited inbound traffic you never asked for. This connection awareness makes it more useful, and it's the modern default for most purposes.

An **application-layer** firewall (or proxy) goes deeper still: it understands the actual protocol (the structure of an HTTP request, for instance) and can make decisions based on the *content*, not just the

addresses. A web application firewall that blocks a malicious-looking request is working at this level. Deeper inspection has processing and configuration costs. Capabilities and performance depend on implementation, and encrypted content may be unavailable unless the design explicitly provides authorised inspection.

8.4 Writing the rules

The single most important decision in a firewall policy is its **default stance**, and it comes in two flavours. **Default-deny** blocks everything and permits only what you explicitly allow: a guest list. **Default-allow** permits everything and blocks only what you explicitly forbid: a banned list. Default-deny limits unexpected access at a defined boundary: with a banned list you can only block the threats you *thought of*, and attackers make their living on the ones you didn't. A guest list fails safe: anything unforeseen is denied by default. A banned list fails open, letting the unforeseen straight through. The trade-off is friction: default-deny means every legitimate need must be anticipated and allowed, which is more work and more support tickets. Test necessary service use as well as prohibited access.

Two more subtleties bite people constantly. **Order matters:** in the first-match model used for our paper exercise, rules are evaluated top to bottom, so a broad "allow" placed above a specific "deny" silently defeats the deny. The packet matches the allow first and is waved through before the deny is ever considered. And **specificity matters:** one rule written too broadly can open a hole nobody notices, and a large rule set becomes so complex that mistakes hide in it. The cost of a bad rule is not abstract; a single misordered or over-broad line is how real networks end up exposed while everyone believes the firewall has them covered.

8.5 VPNs: a private tunnel over a public network

A firewall controls *what can pass*. A **VPN** (virtual private network) protects *what's already passing*. It solves the plaintext problem from the last chapter by building an encrypted **tunnel** between configured endpoints. Traffic selected for that tunnel is protected until it reaches the other endpoint. A full-tunnel configuration and split tunnelling cover different traffic; do not assume every application, DNS request or fallback path is included. To anyone listening in between (the attacker on the coffee-shop Wi-Fi, the operator of an untrusted network), your traffic is just ciphertext. The VPN gives you confidentiality and integrity across a network you don't trust, which is what an untrusted wire otherwise denies you.



Figure 8.2: The problem a VPN solves: a client in the office reaches a server in the datacentre across a path an eavesdropper can read. The tunnel turns that middle section into ciphertext. This illustration joins office `10.11.0.0/24` to datacentre `10.11.1.0/24`; it is not an instruction to connect a lab.

A secure VPN protocol authenticates peers, establishes keys and protects selected traffic against disclosure and undetected alteration. The mechanisms vary by protocol and configuration; some use pre-shared credentials. The cryptography chapter explains why authentication and integrity matter alongside confidentiality.

8.6 What a VPN does not do

VPNs are also the most over-sold tool in consumer security, so it's worth being precise about their limits.

A VPN protects the link between you and the VPN server. It does **not** make you anonymous: your traffic still emerges from the VPN server toward its real destination, and that destination still knows what you did there. It does **not** secure your device: malware already on your machine is unaffected by an encrypted tunnel. And it does **not** protect the leg *beyond* the VPN server. If that onward traffic is unencrypted, it's exposed again once it leaves the tunnel. A VPN is not a substitute for HTTPS. Whether a separate VPN is useful depends on the access requirement and threat model. A VPN also doesn't remove trust; it *relocates* it. You've stopped trusting the local network and started trusting the VPN provider, which sees routing information and any unencrypted onward traffic. Properly protected HTTPS content remains encrypted to the website; leaving the VPN tunnel does not itself reveal that content. That can be a good trade on hostile Wi-Fi and a bad one with a shady provider, but "a VPN makes you safe" is a marketing line rather than a security fact.

8.7 Defence in depth, again

Consider what each tool leaves outside its scope. A firewall controls what traffic is allowed but can't judge the intent behind traffic it permits. Malware happily tunnels out over the same port 443 your firewall allows for ordinary web browsing. A VPN protects traffic in transit but does nothing about what happens at either end. So they join the layers from the last chapter: the firewall decides what's allowed, the VPN protects what's in motion, segmentation limits how far a foothold spreads, and monitoring catches the malicious traffic that slips through a permitted door. Check that these controls fail independently where possible, and test the paths between them. More layers do not help if they share the same unchecked administrator access.

8.8 Where this connects

A firewall and a VPN are the two defences against the scanning, sniffing, and man-in-the-middle attacks of the previous chapter, **network security**. A secure tunnel applies the key and authentication ideas from **cryptography**. And the monitoring that backs up a firewall feeds the detection stage of incident response in **incident and disaster planning**.

8.9 Questions to consider

1. Should a firewall **default to deny** or **default to allow**? Argue the trade-off, and say which you'd choose for a company network and why.
2. A VPN "makes you anonymous / makes you safe." What does it actually protect, and what does it *not*?
3. Give one example of a second layer catching something a firewall alone would miss.

8.10 Test both directions of the policy

In 15 minutes, trace all seven flows in Harbour's paper firewall exercise. Explain the rule that decides each one. Repair the policy and repeat the tests, including legitimate service use. Success means four expected allows and three expected denies, with no broad rule defeating a later restriction. This notation is for reasoning only, not deployment.

Case inputs and checks: Harbour dossier.

Chapter 9

Web Security & Data Protection



A web request should expose only the data and actions its user is authorised to access.

Harbour lets customers view bookings through a browser. A mistake in how the application handles a request could expose someone else's information. This chapter connects web weaknesses to access

controls and personal-data responsibilities. A vulnerability can lead to a privacy incident; discovering a flaw alone does not establish that personal data was compromised or notification is required.

9.1 What this chapter covers

By the end you should be able to:

- Explain the most common web vulnerabilities (injection, **XSS**, broken authentication, session issues) at a working level.
- Trace how a web attack unfolds, step by step.
- Explain core **data-protection** obligations (privacy principles and mandatory breach notification) with the major regimes as examples.
- Connect web weaknesses to the data-protection consequences when they're exploited.

9.2 Why the web is the front line

A web application is, almost by definition, exposed: it exists to accept requests from strangers over the internet. Every input field, URL, and uploaded file is a place where untrusted data enters your system, and the browser on the other end is a universal client you don't control. One recurring weakness is **trusting input that should never be trusted**. Another is failing to check what an authenticated user is allowed to do. Data arriving from a user is treated as if it were safe, and the attacker's craft is to make that data do something it was never meant to.

9.3 Injection: when data becomes commands

The archetype is **injection**, and its most famous form is **SQL injection**. A web application builds a database query using something the user typed, a search term or a login name. If the application glues the user's input into the query text, an attacker can type input that becomes *part of the command itself*. Instead of a username

they submit a fragment that changes what the query does: turning "find the user named X" into "find every user," or worse, "delete the table." The database, unable to tell the developer's intent from the attacker's addition, obeys.

Injection generalises beyond databases. Any time untrusted input is fed into something that interprets commands (a database, the operating system, a template engine), the same flaw can appear. And the defence generalises too: **keep data and code separate.** Rather than build a command by pasting text together, use mechanisms (parameterised queries, prepared statements) that hand the input to the system explicitly *as data*, so it can never be mistaken for instructions. Validate what comes in, and give the application only the database privileges it needs, so a successful injection does less damage.

9.4 Cross-site scripting: running attacker code in your browser

Cross-site scripting (XSS) is injection aimed at the *other* users of a site rather than its database. It is worth understanding because it explains how an attacker ends up running code in *your* browser without ever touching your machine.

A site lets users submit content (a comment, a profile, a message) and displays it back to other users. If the site fails to treat that content as untrusted, an attacker can submit a small **script** instead of ordinary text. When another user views the page, their browser receives that script *as part of the trusted site's page* and runs it in the site's context, with the victim's session. Now the attacker's code, executing as if it were the site itself, may read an accessible session token, act as them, redirect them, or deface what they see. The victim did nothing wrong except visit a page they had every reason to trust. The defence mirrors injection: treat all user content as untrusted, and **encode it for the output context** so the browser treats it as data. HTML text, attributes, URLs and scripts have different rules; safe framework facilities and sanitisation are needed

where rich content is deliberately allowed. OWASP's XSS guidance (https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html) explains why one universal escaping function is insufficient.

9.5 Authentication and session weaknesses

The web has a structural quirk that creates its own class of attacks. The protocol underneath is **stateless**: each request stands alone, with no memory of the last. So after you log in, the server issues a **session token** (usually stored in a cookie) that your browser sends with every subsequent request to say "it's still me." That token *is* your authenticated session. Anyone who steals it can impersonate you **without ever knowing your password**. This is **session hijacking**, and it's why the sniffing and XSS attacks in this book are so dangerous: they're often after the token rather than the password. A related trick, **session fixation**, has the attacker plant a session identifier they already know, then wait for the victim to log in under it. The defences are concrete: send tokens only over HTTPS so they can't be sniffed, flag cookies so scripts can't read them and they don't leak, issue a fresh token at login, and expire and revoke sessions so a stolen one does not remain useful indefinitely. An HttpOnly cookie prevents JavaScript reading that cookie; injected code may still cause authenticated actions. Cookie flags do not make an XSS flaw harmless.

9.6 A valid login is not permission

Suppose Harbour's customer page accepts a booking identifier. A customer changes that identifier and receives another person's booking. The login may be genuine and the query parameter safely handled; the missing check is whether this customer is allowed to access that record.

Enforce ownership and role checks server-side on every relevant request. Hiding a button is not access control.

Test both a permitted action and an attempt by a different account that should be denied. OWASP's access-control guidance (https://top10.owasp.org/2025/A01_2025-Broken_Access_Control/) describes this distinction.

9.7 The other side: protecting the data

Behind every one of these attacks is *data*. Increasingly it is personal data, and the law now requires its protection. Modern **data-protection** law sets out principles for handling personal information that recur across jurisdictions: collect only what you need and only lawfully; use it only for the purpose you stated; keep it accurate; **secure it**; don't retain it longer than necessary; and let people see and correct what you hold about them. Australia codifies these as the **Australian Privacy Principles (APPs)** under its Privacy Act; the EU's **GDPR** is another version of the same pattern; other countries have their own. The names differ; the shape is consistent. The point for a defender is that *security is written into the law*. The applicable duties depend on the organisation, data and jurisdiction. A breach warrants investigation; its occurrence alone does not establish that every relevant legal duty was violated.

9.7.1 When it breaks: breach notification

A technical finding starts an assessment, not an automatic notification verdict. Identify the information, the affected people, what occurred, and which laws cover the organisation. Record uncertainty and involve the responsible privacy adviser promptly.

Australia: NDB scheme. For covered entities, an eligible breach generally involves unauthorised access or disclosure, or loss likely to lead to it, where serious harm to an individual is likely. Successful remedial action and exceptions can affect notification. Suspected eligible breaches need assessment: take all reasonable steps to complete it within 30 calendar days. Once there are reasonable grounds to believe an eligible breach occurred, notify the OAIC and relevant individuals as soon

as practicable. Thirty days is not a general waiting period. OAIC's guidance (https://www.oaic.gov.au/privacy/notifiable-data-breaches/preventing-preparing-for-and-responding-to-data-breaches/data-breach-preparation-and-response/part-4-notifiable-data-breach-ndb-scheme)

EU: GDPR. A controller notifies the supervisory authority without undue delay and, where feasible, within 72 hours of becoming aware of a personal-data breach, unless it is unlikely to pose a risk to people's rights and freedoms. A likely *high risk* generally requires communication to affected individuals without undue delay, subject to exceptions. Processors notify the controller without undue delay. Document breaches and decisions even where regulator notification is not required. EDPB's notification guidance (https://www.edpb.europa.eu/documents/guideline/guidelines-92022-on-personal-data-breach-notification-under-gdpr_en)

These educational summaries were checked in September 2026. They are not a complete statement of the law; other sectoral or contractual obligations may also apply. For a real incident, use the current rules and authorised legal advice.

9.8 From a web flaw to a decision

A report that one customer can read another's booking establishes an access control problem. Logs showing an unauthorised bulk export add evidence of exposure. Whether notification is required still depends on coverage, information involved, risk or harm thresholds and remedial action.

At Harbour, preserve the export evidence and escalate to the privacy adviser. Do not equate a successful restore with resolution of disclosure, or a lack of actor attribution with a lack of harm. The task is to make a supported, timely decision while technical investigation continues.

9.9 Where this connects

This chapter connects to several others. **Authentication** is central because the session tokens attacked here are how the web remembers you are logged in, and stealing one sidesteps the password. **Network security** matters because sniffing and man-in-the-middle attacks are common ways session tokens get stolen, which is why HTTPS matters. **Incident & disaster planning** shares the breach-notification clock here, the same one that drives a real breach response.

9.10 Questions to consider

1. In plain words, how does a **cross-site scripting** attack end up running an attacker's code in *your* browser, on a site you trust?
2. A company is breached and customer data leaks. Under a mandatory breach-notification scheme, what are they obliged to do, and why does that obligation exist at all?
3. If an attacker steals your active **session token**, what can they do that they couldn't with just your username, and what, if anything, can they still *not* do?

9.11 Separate three decisions

In 15 minutes, distinguish a reported access-control flaw, the supplied bulk-export evidence, and a legally notifiable breach. Record what Harbour's privacy adviser knows and must still establish. Then contrast the Australian and EU paths using the sources above. Success means no automatic 72-hour rule for every incident, and no claim that a restore resolves disclosure.

Case inputs and checks: Harbour dossier.

9.12 Take it to the AI

Optional: attempt the paper task first. Use only the fictional case, never real messages, credentials, personal information or internal logs. Check any external claims against the primary sources; do not deploy generated rules.

> Act as a reviewer of an educational notification decision, not as my legal adviser. Use Harbour's fictional facts. Separate what is known, what remains uncertain, and which current regulator source addresses the uncertainty. Do not turn a vulnerability report into proof of a notifiable breach. Ask me to justify my decision before offering your assessment.

Part IV

Part IV · People & the Evolving Threat

Chapter 10

Cybercrime, Botnets & Forensics



Many attacks are a business. Understanding who profits, and how, helps explain the choices defenders face.

Many financially motivated attacks involve a market of specialists selling access, malware and services. Understanding that market helps explain why a compromised ordinary device can matter. Other

adversaries may seek espionage, disruption or personal harm, so financial incentives are one lens, not a complete threat model.

This chapter uses the malware vocabulary from chapter 2, then concentrates on evidence: what an observation supports, what it merely suggests, and what additional information an investigator would need.

10.1 What this chapter covers

By the end you should be able to:

- Explain what a **botnet** is, and how one is built and monetised.
- Describe the **cybercrime economy**: the division of labour behind modern attacks.
- Explain, at a high level, how a real botnet works (credential-stealing and IoT examples).
- Connect forensic **traces** to the attribution of an attack, and explain chain of custody.

10.2 Cybercrime is a business

The single most useful shift in thinking is to treat serious cybercrime as *commerce*. Attackers have costs, revenues, competitors, and customers. They reinvest profits, offer support, build reputations, and specialise. Specialisation is more efficient, just as in any legitimate economy. Cryptocurrency can help move value across borders, but public-ledger transactions may also provide an investigative trail. Traceability depends on the system and available evidence. The result is a mature underground market where almost any capability can be bought.

Seeing crime as business reframes defence. You don't have to make an attack *impossible*. For opportunistic profit-driven attacks, raising costs and reducing returns can discourage targeting. That does not guarantee deterrence. Raise the attacker's cost, lower their expected return, and much of the volume moves to an easier target. A great deal of security is about not being the cheapest victim on the market.

10.3 The division of labour

The defining feature of the modern cybercrime economy is that **participants can specialise rather than doing the whole job.** The work is split among specialists, each selling to the next. **Malware developers** write and sell the tools, often as polished kits with updates and customer support. **Initial access brokers** specialise in breaking in, then *sell the foothold*, access to an already-compromised company, to whoever wants to use it. **Infrastructure providers** rent out "bulletproof" hosting and the command servers that run attacks. **Ransomware-as-a-service** operators lease their ransomware to affiliates, who run the attacks and hand back a cut. **Money mules and launderers** turn stolen funds into clean, spendable cash.

This specialisation has three consequences, and they explain why cybercrime is so stubborn. It **lowers the barrier to entry**: you no longer need to be a skilled hacker to launch an attack, you can rent the capability. It makes the ecosystem **resilient**: arrest one malware author and the access brokers, hosts, and launderers all carry on, and a competitor fills the gap. And it **muddies attribution**: with so many hands touching an attack, working out who is *responsible* is hard. A disruption may leave other participants able to rebuild or reuse parts of the operation.

10.4 Botnets: crime at scale

Chapter 2 introduced a **botnet**: compromised devices under remote control through a command-and-control channel. Here the important distinction is between a device's own data and its value as infrastructure. A device can supply traffic, computing capacity or a relay even when its owner believes it has nothing worth stealing.

A single device can still be valuable, for example because of its accounts or network position. Scale adds other uses such as distributed denial of service. A defender should consider both individual compromise and the harm a fleet can cause to others.

10.5 How they work: two patterns

Two well-known cases show the pattern without needing any perishable statistics.

Gameover Zeus, a member of the Zeus family, was used for credential theft and fraud. The FBI's 2014 account of its disruption (https://www.fbi.gov/news/stories/gameover-zeus-botnet-disrupted) shows why a response can involve investigators, service providers and affected device owners. Disrupting infrastructure and cleaning infected devices are different parts of the work.

Mirai recruited internet-connected devices, including cameras and routers, using weak or default credentials. The US Department of Justice's account of the 2017 guilty pleas (https://www.justice.gov/archives/opa/pr/justice-department-announces-charges-and-guilty-pleas-three-computer-crime-cases-involving) provides a historical source. For a defender, unmanaged devices and default credentials belong in the asset inventory even when their owners do not think of them as computers.

10.6 Following the trail: forensics and attribution

When an attack succeeds, the work shifts to investigation: **digital forensics**, the discipline of reconstructing what happened from the traces left behind. And attackers leave more traces than they'd like. System and network **logs** may record activity. File **timestamps** and **document metadata** can suggest a sequence, but can be modified. Some **email headers** are trustworthy only as far as the systems that added them; sender-supplied fields can be forged. Check clock offsets, collection context and corroborating observations. From fragments like these an investigator builds a **timeline**: a defensible account of who did what, and when.

Two ideas make forensics a discipline rather than a rummage. The first is **chain of custody**: evidence handling should make integrity and provenance open to examination: who touched it, when, and

how, in an unbroken record. Poor handling can undermine confidence and admissibility. A digest shows whether a captured item changed relative to a recorded value; it does not prove the original item was truthful or identify its author. (This is the hard lesson the game module makes you feel.) The second is that **attribution is difficult**. The same division of labour and the same botnets that power attacks also hide the people behind them: traffic routed through a botnet appears to come from innocent victims, infrastructure is rented and disposable, and false clues are sometimes planted deliberately. Careful defenders talk about attribution in terms of *confidence*, not certainty.

Choose the next investigative step for its relevance and reliability, not because one kind of trace is always the clue attackers forget. Preserve the supplied material and document collection, access and interpretation. Investigation supports containment and recovery as well as later attribution; it need not wait until systems are restored. NIST's incident-response guidance (https://csrc.nist.gov/pubs/sp/800/61/r3/final) provides the wider operational context.

> Play it
>
> *Incident Zero*'s **Forensics** module has you chase attribution and rebuild a timeline while protecting the chain of custody. Mishandle the evidence trail and you lose the case, even when you know who did it. □ incidentzero.retroverse.studio (https://incidentzero.retroverse.studio/)

10.7 Where this connects

This chapter connects to several others. **Software security & malware** supplies the weapons this economy trades in: the trojans, worms, and ransomware from that chapter. **Network security** matters because a botnet's command-and-control and DDoS traffic are what network monitoring exists to spot. Forensics supports the containment and recovery decisions in **incident and disaster planning**,

as well as subsequent review.

10.8 Questions to consider

1. Why is a **botnet** so much more valuable to a criminal than a single compromised machine?
2. Modern cybercrime has a "division of labour": different people write malware, sell access, rent servers, and cash out. Why does that specialisation make attacks so hard to stop?
3. Choose one **trace** from Harbour's evidence cards. What does it support, what can it not establish, and what would you check next?

10.9 Annotate an evidence trail

In 15 minutes, annotate Harbour's evidence cards as observation, inference or unknown. Explain what supports concern about an export and why an account name is not an actor identity. Write a handling record with collector, time, item, digest reference and access history. Success means a defensible timeline with uncertainty visible, not a confident name to blame.

Case inputs and checks: Harbour dossier.

Chapter 11

Human Factors & Emerging Trends



A useful defence makes mistakes easier to report and less costly to survive.

Security depends on people using and maintaining systems under ordinary working pressures. A login prompt, an urgent request or an unclear warning can put the wrong decision within easy reach.

The design question is what happens next: can the person get help, and do other controls limit the harm?

This chapter brings together usable procedures, independent verification and a cautious assessment of AI's effects. The final chapter applies those ideas to the complete defender's dossier.

11.1 What this chapter covers

By the end you should be able to:

- Explain how **human** decisions interact with system design, and why blame alone does not repair a weakness.
- Describe **social engineering** and **phishing**, and the defences that actually work.
- Reason about how non-technical people make security decisions with the information they're given.
- Discuss **emerging trends**, including AI's double role as both a tool for attackers and a tool for defenders.

11.2 The weakest link, and why blaming the user fails

Calling people the weakest link treats a complex system as a single chain. More usefully, ask what the interface, permissions and work process made likely. The claim that "the users are the weakest link" is a *trap*: a framing that excuses bad security. When a well-crafted phishing email fools an employee, the instinct is to blame the employee: they should have known better. But look closer. They were doing their job, quickly, inside a system that made the wrong action easy, the right action unclear, and the difference between the two nearly invisible. Blaming the person lets the *system* off the hook. A team that blames users stops fixing the thing they could fix.

A more useful stance examines the system. People will make mistakes. That is a constant, and good security is designed around it rather than trying to train it away. This is *assume breach* applied to

human beings: assume that, sooner or later, someone will be fooled, and build so the organisation survives it. That single shift, from "make users perfect" to "make mistakes survivable", is the difference between security that works and security that just apportions blame after the fact.

11.3 Social engineering and phishing

Social engineering is hacking the human instead of the machine: manipulating a person into doing what the attacker wants. It works because it targets the mental shortcuts we all rely on to get through the day. An attacker invokes **authority** ("this is IT, we need your password"), **urgency** ("the account will be closed in an hour"), **fear**, **trust**, or plain **helpfulness**. The victim, trying to be efficient and cooperative, complies. They are usually behaving *well* by the ordinary rules of work; the attacker has pointed those good instincts in the wrong direction.

Phishing is social engineering at scale: the deceptive email or message, cast wide, that lures people into revealing credentials or running malware. **Spear-phishing** is the targeted, personalised version aimed at a specific person, and it is more convincing. These attacks succeed because they exploit how humans are built to work. Their victims are not foolish.

The defences that help are instructive, because the obvious one is the weakest. **Awareness training** matters, but you cannot train people to be perfect forever, and a defence that requires every human to be vigilant every time will eventually fail. The stronger approach is to *reduce the reliance on human vigilance*: use technical controls that make a mistake less catastrophic. Multi-factor authentication means a phished password alone isn't enough, which is the point of the authentication chapter. Email filtering can remove some lures before they reach a person. Firm *verification procedures* for high-risk actions mean no single email can move money or hand over data without an independent check. Best of all is to **make the safe path the easy path**, so that doing the secure thing is also the convenient thing. The goal is a system in which an ordinary human error does

not become a disaster, rather than a workforce that never errs.

11.4 Communicating risk to people who aren't technical

A quieter human-factors failure is that we push security *decisions* onto people who have no way to make them well. The classic example is the browser **certificate warning**. When your browser says a site's certificate can't be trusted, it is telling you something important. It could not verify you're talking to who you think you are, using the identity-proving cryptography from the cryptography chapter. That warning might be the only thing standing between you and a man-in-the-middle attacker.

Some users bypass warnings when the language is unclear, the warning interrupts urgent work, or past experience has taught them to discount it. Behaviour varies with the warning and context; do not assume most users always ignore it. This is a *design* failure, not the user's. A consequential security decision has been handed to someone without the information or context to make it, in words that mean nothing to them. Good security gives people clear information and, wherever possible, doesn't force impossible judgements on them.

11.5 Emerging trends: AI on both sides

AI can assist both attackers and defenders, but capability claims need a date, a task and evidence. It can lower the effort of drafting messages, summarising data or producing code. That does not make every message flawless, every exploit effective or an operation limitless.

For attackers, plausible personalised language and synthetic audio can make familiar verification shortcuts less reliable. For defenders, models may help triage reports and interpret logs, but false alarms, missed attacks and untrusted inputs still need evaluation. Logs and messages can contain hostile instructions as well as sensitive

information, so an assistant must not be given authority merely because it produced a confident answer.

The NCSC assessment of AI and cyber threats to 2027 (https://www.ncsc.gov.uk/report/impact-ai-cyber-threat-now-2027) describes expected changes in attacker capability. Treat it as a dated assessment with uncertainty, not proof that every forecast has occurred.

At Harbour, an apparent manager's voice is not enough to approve an account reset or bulk export. Use a known contact route and the agreed approval process. This remains useful whether the message was written by a person or generated by a model. Give staff a reporting path that does not punish them for raising a concern that turns out to be harmless.

11.6 Where this connects

This chapter connects back across everything before it. Human decisions influence the design, operation and recovery of technical controls, so human factors belongs throughout the book. Authentication is the clearest case: MFA is a technical control that makes a human mistake survivable. And that same thread, the human running through every stage, is part of how the next chapter ties the whole book into a single lifecycle.

11.7 Questions to consider

1. "Users are the weakest link." Argue *against* this framing: whose fault is a successful phish, really?
2. What does a browser certificate warning tell you? Explain why someone might bypass it and propose a safer route the organisation could offer.
3. Give one way AI helps **attackers** and one way it helps **defenders**. Which side do you think it favours, and why?

11.8 Make reporting usable

In 10 minutes, write a response to this fictional message: “Mara here. Approve the login prompt now so I can finish the bookings.” Use a known contact route and the agreed approval process. Add a non-blaming report for someone who already approved it. Success means prompt escalation and independent verification, without using contact details supplied by the message.

Case inputs and checks: Harbour dossier.

11.9 Take it to the AI

Optional: attempt the paper task first. Use only the fictional case, never real messages, credentials, personal information or internal logs. Check any external claims against the primary sources; do not deploy generated rules.

> Act as a reviewer of Harbour’s reporting procedure. Use only the fictional message and case facts. Ask me what someone who already approved a prompt should do next. Then identify one obstacle in my procedure and one change that makes reporting easier without bypassing verification.

Part V

Part V · Putting It Together

Chapter 12

The Long Game



A defence plan should make its limits as visible as its controls.

You now have the pieces of a defender's dossier. The final test changes the budget and recovery target. It asks whether you can recognise an unresolved risk and explain it to the person authorised to decide. A polished plan that hides a failed test is less useful than a short one that exposes it.

12.1 What this chapter covers

By the end you should be able to:

- Connect assets, controls, evidence and recovery decisions in one plan.
- Check whether a changed requirement invalidates an earlier answer.
- Distinguish a justified decision from a claim that everything is secure.
- Assign responsibility for unresolved risk and future review.

12.2 Joining the dots

Return to the seven activities introduced in chapter 1: design, harden, detect, respond, recover, investigate, and audit/learn. Human decisions run through every activity. These are overlapping tasks, not seven departments that hand an incident along in a fixed sequence.

At Harbour, permissions influence whether a login can become a bulk export. Logging affects what the response lead can establish. Recovery credentials affect whether backups survive. A notification assessment may still be open after the service is restored. Those connections are what the dossier should make visible.

A checklist can help an authorised responder avoid missing a necessary step. It needs an owner, a context and testing. The failure is treating checked boxes as proof that the service can meet its obligations under changed conditions.

12.3 Complete the independent case

Use Harbour's weekend programme. Allow 30–45 minutes, working alone without AI. The appendix supplies every changed fact and an eight-point check. Write your answer before reading the comparison.

Your one-page decision needs the assets and permissions, an affordable control choice, risk arithmetic, allowed and denied tests, both recovery checks, an evidence-preserving response and a privacy escalation. End with what is ready, what is not, who can decide, and what evidence is needed next.

You are not required to make the constraints fit. If the stated recovery time misses the target or the budget leaves a known boundary weakness, say so. Do not create an imaginary free control to make the answer look complete. An explicit request for a changed budget, a tested improvement or a deferred launch is a legitimate result.

12.4 Check and transfer

Compare your answer with the reasoning check. Aim for seven of eight criteria, with no unsafe testing, unsupported readiness claim or missed recovery-target failure. If you disagree with the worked choice, identify the constraint and evidence that make your alternative defensible.

For a second attempt, ask another reader to change one supplied constraint. Predict which section of your dossier must change before recalculating it. That is a stronger test of understanding than repeating the original answer.

> **Play the whole thing**
>
> Every stage in that loop is easier to *understand* than to *do*. Doing it, under time and budget pressure, is where the judgement lives. *Incident Zero*, the free print-and-play game referenced throughout these chapters, has a full campaign that runs this entire lifecycle end to end: design a network, harden it, detect and respond to a breach, recover from the disaster, and investigate what happened. Playing it is the closest thing to *living* a security cycle in an afternoon. If this book gave you the map, the campaign lets you walk the ground. □ incidentzero.retroverse.studio (https://incidentzero.retroverse.studio/)

The game is optional group enrichment. Its current landing page (https://incidentzero.retroverse.studio/) describes the player, printing and time requirements. The paper dossier above is a complete solo route and does not reproduce the game's materials.

12.5 Questions to consider

1. Which supplied fact most changed your control choice, and why?
2. Which conclusion depends on evidence you do not yet have?
3. What should trigger a new review after an initially acceptable plan?

12.6 Carry the decision forward

Keep the dossier as a record of a decision under stated conditions. When the system, staff, evidence or obligations change, revisit it. The useful habit is to explain what a control does, demonstrate it where possible, and make the remaining uncertainty visible to the right decision-maker.

Appendix: Setting Up a Practice Environment

The complete core route is the paper-based Defender's Dossier. Its observations and rules are fictional. You can learn the reasoning without scanning a device, cracking a password or capturing anyone's traffic.

Permission before practice

For optional hands-on work, obtain explicit written permission for the systems, accounts, techniques and time window involved. Owning a laptop does not authorise testing the shared network, a cloud provider or other users. If scope is unclear, stop and ask the responsible owner.

Use synthetic data and test accounts. This book does not ask you to handle live malware, attack a public website or upload real logs to an assistant.

Isolation is a property to check

A VM is a separate guest operating system. A container shares more of its host's underlying environment. Either can be useful; neither name proves that a mistake cannot reach real systems.

With VirtualBox, **NAT allows outbound access to the host/LAN and internet**. **Host-only connects guests to the host**. An

internal network can connect selected guests without attaching that network to the host or outside world. Additional adapters, routing, forwarding and shared resources can change the boundary. These distinctions are explicit in Oracle's networking table (https://docs.oracle.com/en/virtualization/virtualbox/7.1/user/networkingdetails.html).

Before using a deliberately vulnerable environment, check every adapter, published port, shared folder and credential. Keep personal files and production credentials outside it. Use only an environment whose maintainer documents the isolation and reset procedure for your platform. A container with a published port or mounted host directory is not an isolated playground merely because it starts in a container.

A safe optional route

Choose a maintained teaching environment with explicit authorisation and setup instructions. OWASP Juice Shop (https://owasp.org/www-project-juice-shop/) is one example of an intentionally insecure training application; read its own setup and safety documentation before installing anything. Do not treat its public demonstration or someone else's installation as an authorised target.

Write down the intended boundary, confirm what can reach what using the environment's documented checks, and verify its reset procedure before an exercise. If a service is exposed beyond that boundary, a test reveals real data, or you cannot explain a connection, stop. Disconnect the exercise environment and ask its maintainer or your instructor for help.

A snapshot helps reset state; it does not remove data that has already escaped. Record the environment version and your observations so another authorised learner can reproduce the exercise. Tool-specific installation is deliberately outside this book; if you cannot establish the boundary, use the paper route.

Tools by capability

Names provide anchors, not required installations.

Capability	Examples
Inspect authorised host/service exposure	Nmap
Study synthetic password hashes	John the Ripper, Hashcat
Inspect supplied or authorised packet captures	Wireshark
Work with keys and signatures	OpenSSL, GnuPG
Test a permitted firewall policy	nftables
Examine supplied forensic evidence	Autopsy, Volatility

Follow the official documentation and scope for the chosen environment. Successful setup is not the learning outcome: explaining an observation, its limitations and the defensive decision it supports is.

Appendix A

The Defender's Dossier

This is an original, fictional case. All people, records, prices, observations and risk estimates below are invented for teaching. No real system needs to be contacted. A notebook is enough; the tables are the complete exercise inputs, not extracts from a missing lab.

A.1 Harbour Community Centre

Harbour runs public workshops. Eight staff use a hosted booking application. It holds 600 participant records: names, contact details and workshop choices. It holds no payment-card data. A public web service talks to a private booking database. Staff administer the application through a VPN gateway. Guest Wi-Fi should provide internet access without access to the database or staff devices.

Mara, the manager, approves bulk exports and account changes. Rae, the receptionist, needs to view and amend individual bookings, but does not need bulk export or account-administration rights. Volunteers need only the attendance list for their assigned workshop. The current system mistakenly gives every staff account bulk-export permission. Customers should see only their own booking. A shared administrator password is another current weakness.

The centre director owns the risk decision. Mara coordinates an incident; the service provider handles technical containment and

recovery under an agreed scope. A designated privacy adviser determines applicable legal duties. For this exercise assume the centre is covered by Australia's NDB scheme. There is no assumption that every real small community organisation is covered.

The booking service has a recovery time objective of four hours and a recovery point objective of one hour. Disruption starts at 09:20 in the exercise. Backups currently run at 23:00 each night using the application's administrator credentials. A previous restore test took two hours, excluding containment and validation. A successful backup job is not evidence of a successful restore.

A.1.1 A bounded budget

The director can spend **$3,000 per year** on additional controls. These are fictional annual costs including setup amortisation and staff time. Existing basic application logs and staff reporting remain available with any choice.

Option	Annual cost	What it changes
A: account and access repair	$1,000	Individual accounts, administrative phishing-resistant MFA, reviewed permissions, recovery checks and session revocation procedure
B: network boundary repair	$800	Guest isolation, restricted database paths, reviewed firewall policy and allowed/denied connection tests
C: recovery repair	$1,200	Hourly recovery points, separate backup administration, off-site immutable copies and scheduled restore tests
D: managed alert review	$1,000	A staffed alert-review service with an agreed escalation time; does not change access rights or backup protection

A, B and C cost exactly $3,000. D cannot be added to that combination within the budget. The centre must still assign someone to review its existing logs and reports if it does not buy D. None of the options guarantees prevention.

For a *separate* expected-loss calculation, use an incident loss of $50,000 and an estimated annual occurrence rate of 0.10. Suppose option A reduces that rate to 0.04, without changing loss per incident. These are scenario assumptions, not measured product effectiveness.

A.1.2 A paper firewall model

Use zone names rather than real addresses. All rows below describe **new** connections. For this exercise only, the first matching rule decides; an unmatched connection is denied. Return traffic for an allowed established connection is allowed. Do not copy this teaching notation into a firewall.

Rule	Source	Destination	Service	Action
1	Any	Booking database	Any	Allow
2	Guest	Booking database	Any	Deny
3	Public	Web	HTTPS	Allow
4	Web	Booking database	Database	Allow
5	Staff	VPN gateway	VPN	Allow

“Any” includes every named zone and service. The four legitimate new flows are Public to Web using HTTPS, Web to Booking database using Database, Staff to VPN gateway using VPN, and Guest to Internet using HTTPS. All other new flows should be denied in this simplified model. DNS and other supporting flows are deliberately outside the model; a production policy would need them specified and tested.

A.1.3 Evidence cards

These are supplied observations, not a capture or an invitation to investigate a live service. Application events and the manager’s

notes share the same clock in this exercise. Real investigations must check clock offsets.

Time	Supplied observation
09:10	Application log records a successful login to Rae's account from a previously unseen network address
09:12	Application log records a completed export of 600 participant records under Rae's session
09:15	Rae reports approving an unexpected login prompt after receiving an urgent message
09:20	Mara declares a suspected incident and suspends booking access; service disruption begins
09:25	Provider preserves an access-controlled copy of relevant logs, records who collected it and its digest
09:30	Provider revokes sessions and begins a two-hour restoration rehearsal in an isolated recovery environment
11:30	Rehearsal restore finishes; technical checks continue
11:50	Integrity and access checks finish; the rehearsal is accepted

The export event supports investigation of personal-data exposure. It does not identify the person operating the session or establish the destination or subsequent use of the exported records. A new address alone is not proof of compromise. The reported prompt approval is relevant evidence, not a complete explanation. No evidence card proves the source system is clean or authorises reconnecting it.

A.2 A worked dossier

Read this after attempting the chapter tasks. It is one defensible answer, not the only possible policy.

A.2.1 Protection and access

My first asset is participant information. An account with excess export rights could expose many records in one action. I choose individual accounts, restricted exports and an alert on bulk export. The residual risk includes misuse of a legitimately authorised manager account. A permission test should show Rae can update a booking but cannot export all records, and that a volunteer cannot view another workshop's attendance list.

Availability matters when people arrive for a class. Integrity matters when a booking or contact detail changes. Encryption in transit and at rest helps protect disclosure through particular paths; it does not prevent an authorised application session from exporting readable data. A hash stored next to an editable export is not independent proof that the export is genuine. A trusted digest or signature must be checked through a separately trusted channel.

A.2.2 Spending and uncertainty

I choose A, B and C at $3,000. This repairs known access, guest-boundary and recovery weaknesses. I defer D, retain basic logging, and have Mara assign daily log review and an immediate staff reporting route. That leaves a detection-delay risk which the director must review, not forget.

The separate arithmetic for A is:

- Before: $50,000 × 0.10 = **$5,000** annual expected loss.
- After: $50,000 × 0.04 = **$2,000**.
- Reduction: $3,000; less $1,000 annual cost = **$2,000** estimated net benefit.

The break-even post-control rate is 0.08. At 0.09, the reduction is only $500 and the net financial benefit is negative $500. At 0.02 it is positive $3,000. The choice therefore depends on an uncertain estimate. I cannot add up claimed reductions for several overlapping controls as if they were independent. Legal duties and unacceptable harm also constrain the decision, even when expected monetary benefit is negative.

A.2.3 Testing a boundary

The original rule 1 permits Guest to Booking database before rule 2 is considered. It also admits Public to Booking database. Remove that broad permission. A corrected teaching policy allows only the four legitimate flows named above, then denies every other new connection.

Test all four permitted flows and at least these three denied flows: Guest to Booking database using Database, Public to Booking database using Database, and Public to Web using Database. Expected results are four allows and three denies. A policy that blocks everything fails the legitimate-service tests. Testing only a successful website visit would miss the database exposure.

A.2.4 Responding and recovering

Mara can escalate at 09:15 without waiting for an attribution verdict. The provider preserves evidence while containing access, documenting any conflict between preservation and stopping damage. A log digest helps detect later change; the collector, time, storage location and handling record establish context. A digest alone does not prove who created the original log.

The rehearsal completes **150 minutes** after disruption: 10 minutes to restoration start, 120 minutes restoring, and 20 minutes validating. That fits the four-hour RTO in this exercise. It is one test, not a guarantee of future recovery.

With the original 23:00 recovery point, a 09:20 failure could lose **620 minutes** of work, which fails the one-hour RPO. If option C supplies a validated, unaffected 09:00 recovery point, the gap is **20 minutes**. For this comparison assume those recovery points are available and tested. In a real incident, the newest backup might contain compromised data; the provider must establish an acceptable recovery point before reconnecting.

Backups do not undo the export. Mara records what is known, preserves the report, and asks the privacy adviser to assess coverage, information involved, likely harm and remedial action. The adviser

records the applicable notification decision and timing. Neither a successful restore nor an unconfirmed actor identity closes that assessment.

A.2.5 Human process and review

Staff get a known reporting channel for unexpected prompts and requests. A high-risk account reset or export approval uses a second, independent verification route. Staff should not call a number supplied in the suspicious message. The aim is quick reporting and constrained damage, without requiring perfect suspicion from everyone.

The dossier ends with owners and evidence: Mara checks access tests and reporting arrangements; the provider supplies boundary and recovery results; the privacy adviser owns notification advice; the director reviews residual risk and the deferred alert service. Revisit after a permissions change, a failed restore test or a new service, as well as at the scheduled review.

A.3 Independent transfer: weekend opening

Work for 30–45 minutes without AI or the worked answer. All facts stay the same except these changes:

- Budget is **$2,200**. Option prices stay unchanged.
- The director requires **option A** before the new programme opens because bulk-export permissions are unacceptable. This is a supplied constraint.
- The RTO is now **two hours**; the RPO stays one hour.
- Restore takes **100 minutes**, preceded by 10 minutes of response and followed by 20 minutes of validation.
- If C is selected, the latest validated recovery point is 09:00 for a 09:20 disruption. Without C it is still 23:00 the previous day.
- The changed loss estimate is $80,000, initial annual rate 0.10 and estimated rate after A of 0.06.

Write a one-page decision: assets, control choices and annual cost; one permission and one boundary test; expected-loss arithmetic; RTO/RPO checks; incident ownership and evidence; privacy escalation; and the unresolved decision about opening. You may say "not ready". Do not invent a free control, a faster restore, an extra budget or proof that a breach did not happen.

A.3.1 Check your reasoning

A and C use the full $2,200. A and B cost $1,800 but leave the stated backup problem. A and D cost $2,000 but repair neither recovery nor the network boundary. With A required, no affordable combination fixes all three.

For A, the changed expected loss is $8,000 before and $4,800 after; the $3,200 reduction gives **$2,200 net benefit** after its cost. The proposed recovery needs **130 minutes**, so it misses the two-hour RTO by **10 minutes**, even if C meets the RPO at 20 minutes. A+C leaves the network weakness unresolved. A defensible answer escalates the launch decision, identifies the missing boundary repair and demands a tested recovery improvement or an explicitly authorised change of target. It does not quietly claim the service is ready.

Score one point for each of eight checks: correct assets and permissions; affordable choices satisfying the A constraint; correct expected-loss maths; both allow and deny tests; correct RTO and RPO findings; evidence-preserving response with owners; a qualified privacy escalation; and an explicit unresolved launch decision. Aim for at least seven, with no safety failure, no claim that the 130-minute recovery meets a 120-minute target, and no unsupported "ready" conclusion. A different control choice can pass if its limitations and consequences are honestly defended.

Appendix B

Further Reading

This is a route into primary guidance, not a list to memorise. Consult the source for the decision you are making. Standards, product behaviour and legal requirements can change; these links were selected in September 2026.

B.1 Foundations, identity and cryptography

The NIST Cybersecurity Framework (https://www.nist.gov/cyberframework) organises outcomes and governance. ASD's Essential Eight (https://www.cyber.gov.au/business-government/asds-cybersecurity-frameworks/essential-eight) provides a different resource: mitigation strategies and maturity guidance. Neither removes the need to understand the organisation's risks.

NIST SP 800-63B-4 (https://pages.nist.gov/800-63-4/sp800-63b.html) distinguishes authenticators, password verifiers, recovery and sessions. OWASP's password-storage guidance (https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html) explains implementation choices. Read these when assessing a login design, rather than implementing a password store from a textbook sketch.

FIPS 186-5 (https://csrc.nist.gov/pubs/fips/186-5/final) covers digital signatures; TLS 1.3 (https://www.rfc-

editor.org/rfc/rfc8446.html) shows authentication, key establishment and traffic protection as separate operations. They are references for checking explanations, not prerequisites for the paper exercises.

B.2 Software, networks and incidents

The CVE programme (https://www.cve.org/ResourcesSupport/FAQs), CWE catalogue (https://cwe.mitre.org/) and CERT/CC disclosure guide (https://vuls.cert.org/confluence/display/CVD) help distinguish a vulnerability record, a recurring weakness and the process of coordinating disclosure.

OWASP's access-control guidance (https://top10.owasp.org/2025/A01_2025-Broken_Access_Control/) and XSS prevention guidance (https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html) expand the web chapter. Wireshark's Ethernet capture guidance (https://wiki.wireshark.org/capturesetup/Ethernet) helps explain the limits of an observation point.

NIST SP 800-61 Rev. 3 (https://csrc.nist.gov/pubs/sp/800/61/r3/final) connects incident response to risk management. CISA's ransomware guide (https://www.cisa.gov/stopransomware/ransomware-guide) includes preparation, data theft and recovery. NIST's contingency-planning resource (https://www.nist.gov/privacy-framework/nist-sp-800-34) supports recovery objectives and testing.

B.3 Legal decisions and historical evidence

OAIC's NDB guidance (https://www.oaic.gov.au/privacy/notifiable-data-breaches/preventing-preparing-for-and-responding-to-data-breaches/data-breach-preparation-and-response/part-4-notifiable-data-breach-ndb-scheme) and EDPB's GDPR notification guidance (https://www.edpb.europa.eu/documents/guideline/guidelines-

92022-on-personal-data-breach-notification-under-gdpr_en) describe different regimes. Coverage, facts, recipients and thresholds matter. Use authorised advice for a real incident.

The Senate staff report on Target (https://www.commerce.senate.gov/wp-content/uploads/media/doc/2014%200325%20Target%20Kill%20Chain%20Analysis.pdf) is an analysis of reporting available in 2014. Treat its proposed interventions as possible opportunities, not proven counterfactuals. The chapter on cybercrime links the investigating agencies' accounts of Gameover Zeus and Mirai.

The NCSC assessment of AI threats to 2027 (https://www.ncsc.gov.uk/report/impact-ai-cyber-threat-now-2027) is a dated assessment. Separate observed capability, forecast and your own inference when using it.

B.4 Learning beyond the book

The practice appendix explains the scope and safety checks for optional technical work. Incident Zero (https://incidentzero.retroverse.studio/) offers group practice. Its materials have their own licence; this book links them without incorporating them.

For the next security-track step, see *Substantiate, Don't Assume* (https://michael-borck.github.io/substantiate-dont-assume/). For learning conversations, see *Conversation, Not Delegation* (https://michael-borck.github.io/conversation-not-delegation/). Both are optional: the dossier is self-contained.

About the Author



Figure B.1: Photo of Michael Borck

Michael Borck is a software developer, educator, and auditor-by-training working at the intersection of human expertise and artificial intelligence. He teaches information security, and this book grew out of the introductory course he wanted to hand students. It builds the *defender's mindset* rather than a checklist of terms.

His book *Substantiate, Don't Assume* takes the same discipline into professional security auditing. *Conversation, Not Delegation* argues that AI is most valuable as a thinking partner you converse with rather than a tool you delegate to. *Assume Breach* sits at the entry point to that path: the first course, before the audit and before the profession.

Michael builds educational software and resources, and writes about the 80/20 principle in learning and professional judgement.

Connect

- GitHub (https://github.com/michael-borck): Software and open-source projects
- Free book catalogue (https://books.borck.education): Books and learning resources
- 8020workshop.com (https://8020workshop.com): Passion projects and workshops
- LinkedIn (https://linkedin.com/in/michaelborck)

Other Books in This Series

Security Track:

- *Assume Breach: A First Course in Defensive Security* (this book)
- *Substantiate, Don't Assume: Why the Auditor's Opinion Matters More Than Ever in the Age of AI*

AI-Collaboration Track:

- *Conversation, Not Delegation: How to Think With AI, Not Just Use It*
- *Partner, Don't Police: AI in the Business Classroom*

Python Track:

- *Think Python, Direct AI: Computational Thinking for Beginners*
- *Code Python, Consult AI: Python Fundamentals for the AI Era*
- *Ship Python, Orchestrate AI: Professional Python in the AI Era*
- *Converse Python, Partner AI: The Intentional Prompting Methodology*

Web Track:

- *Build Web, Guide AI: Business Web Development with AI*